Моделирование химико-технологических процессов

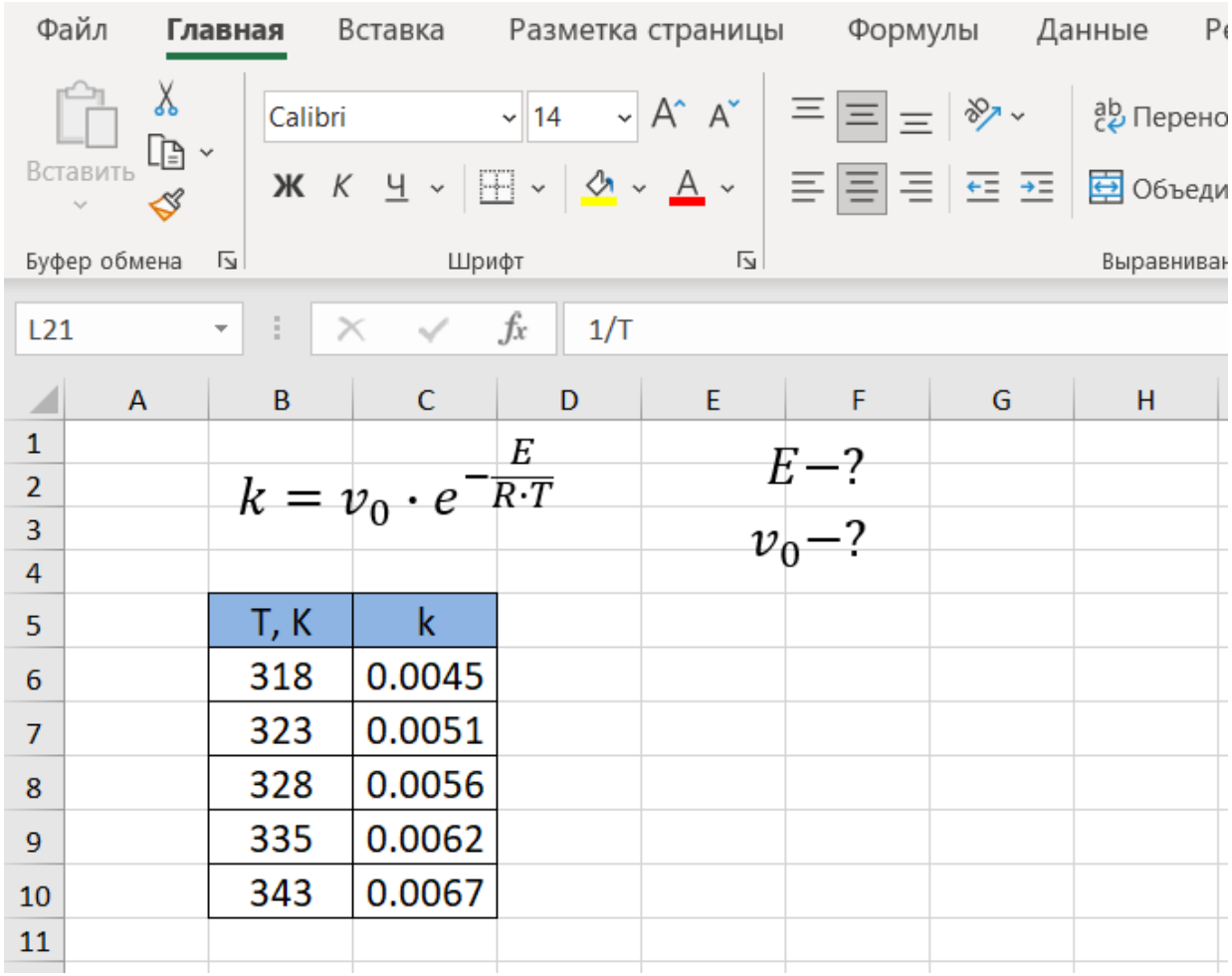
Эмпирические модели и регрессия

Алгоритм построения эмпирической модели

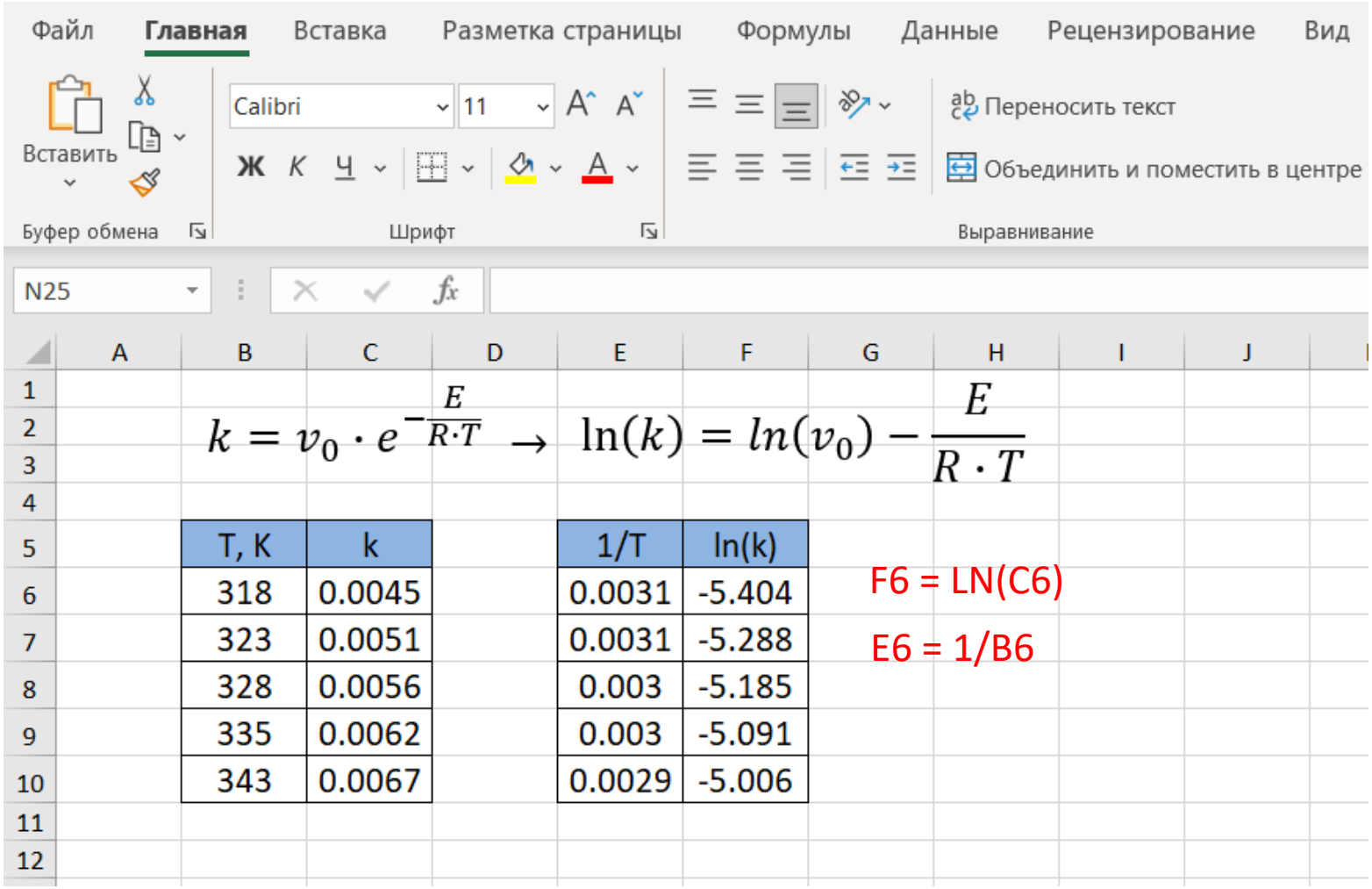
✓ Этапы решения задачи:

- ***проведение эксперимента***
- выбор вида эмпирической зависимости
- нахождение параметров выбранной зависимости
- проверка адекватности модели
- применение модели для прогнозов

Простые эмпирические модели в Excel



Простые эмпирические модели в Excel



$$k = v_0 \cdot e^{-\frac{E}{R \cdot T}} \rightarrow \ln(k) = ln(v_0) - \frac{E}{R \cdot T}$$

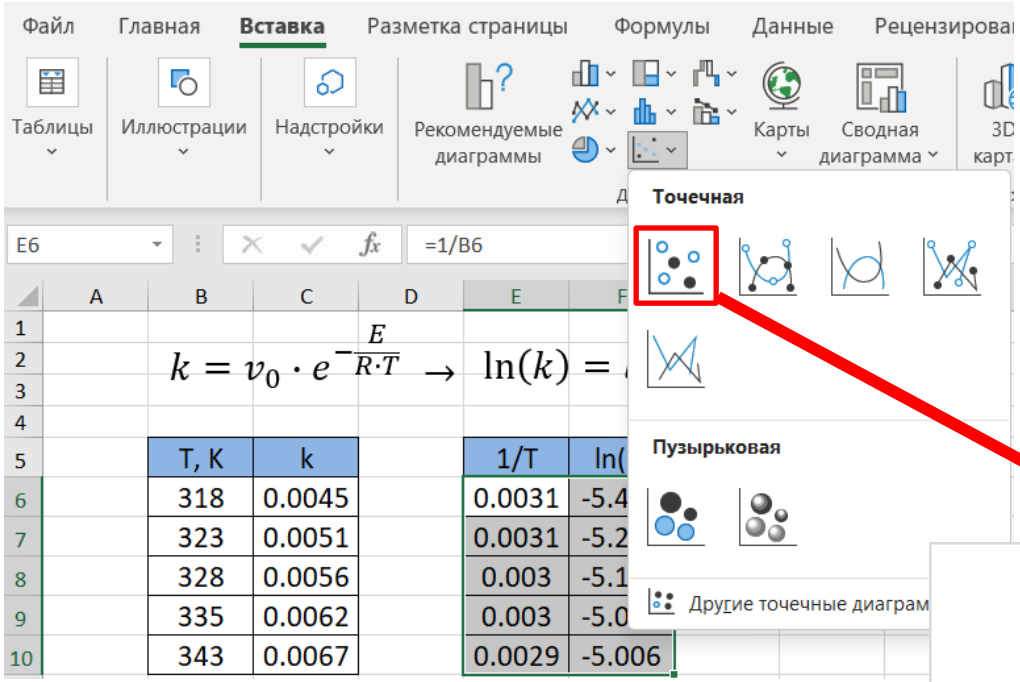
T, K	k
318	0.0045
323	0.0051
328	0.0056
335	0.0062
343	0.0067

1/T	ln(k)
0.0031	-5.404
0.0031	-5.288
0.003	-5.185
0.003	-5.091
0.0029	-5.006

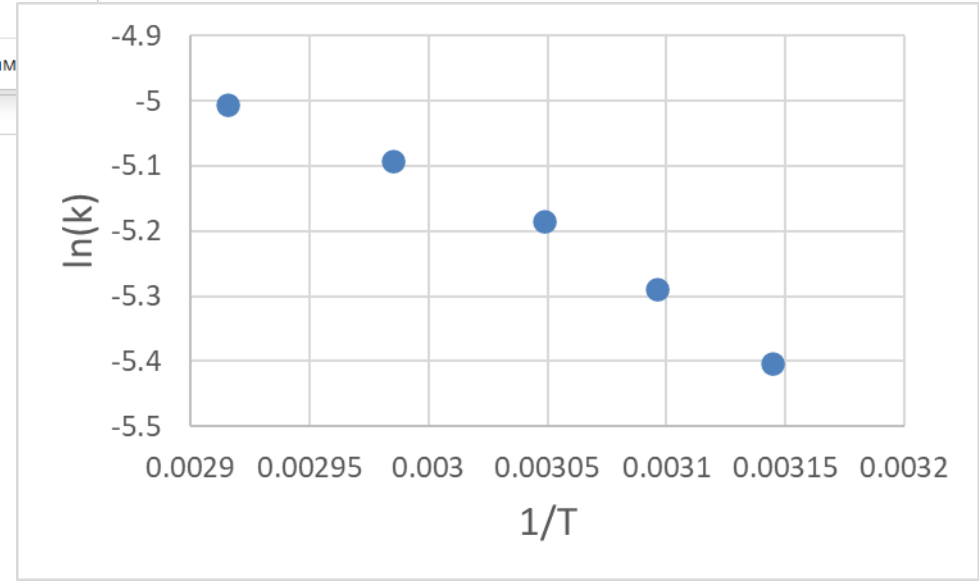
F6 = LN(C6)

E6 = 1/B6

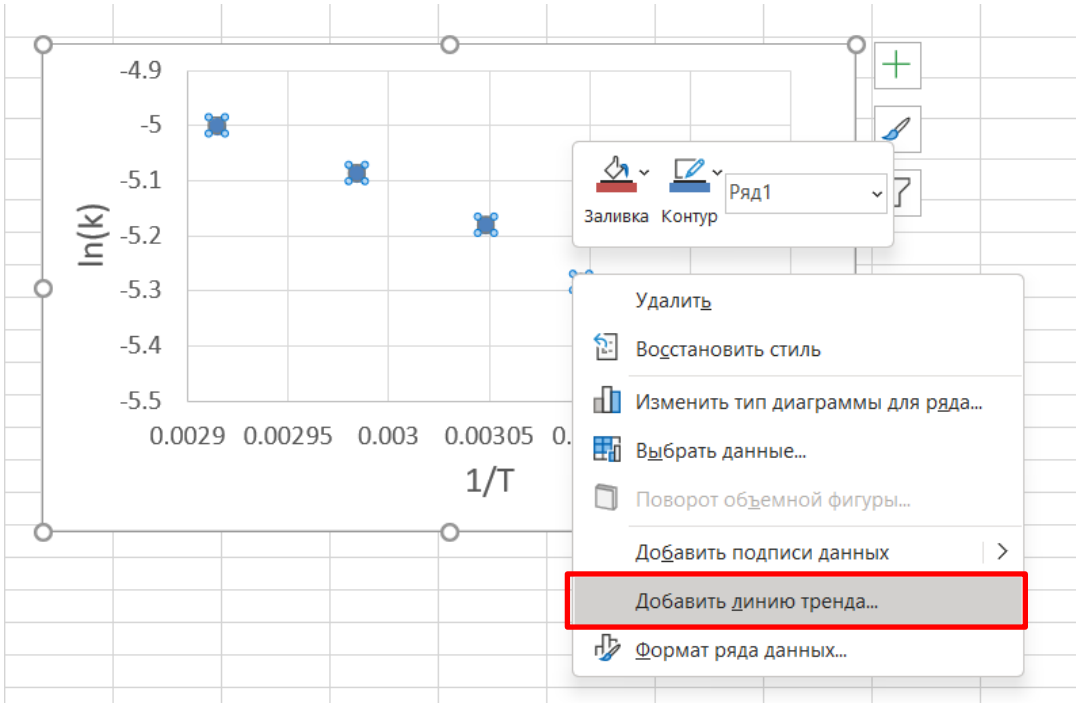
Простые эмпирические модели в Excel

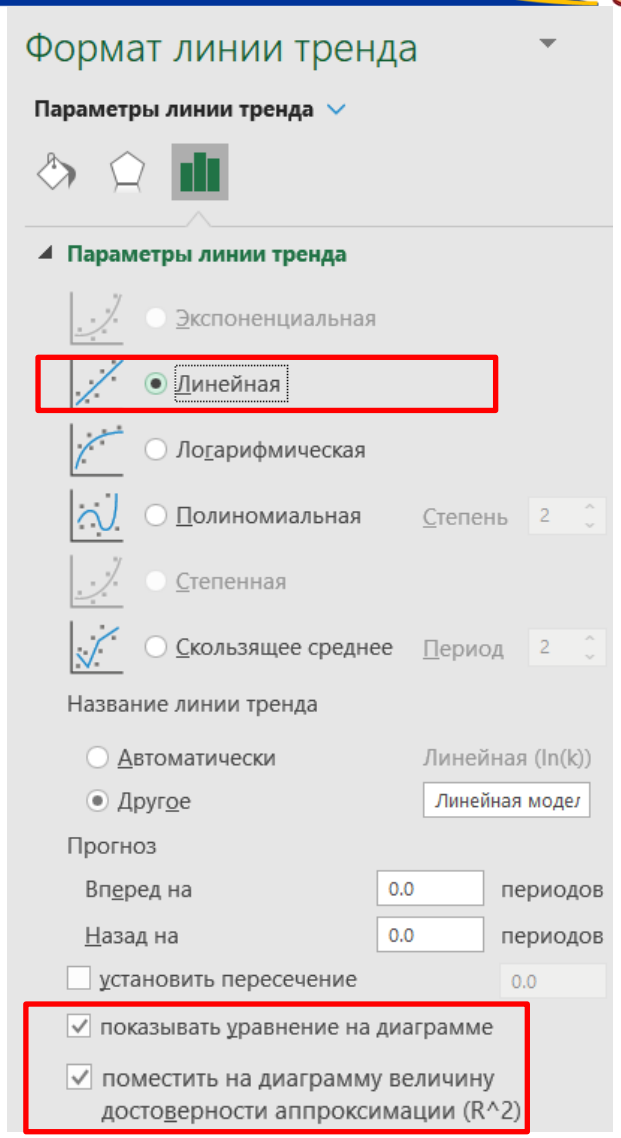


T, K	k	1/T	ln(
318	0.0045	0.0031	-5.4
323	0.0051	0.0031	-5.2
328	0.0056	0.003	-5.1
335	0.0062	0.003	-5.0
343	0.0067	0.0029	-5.006

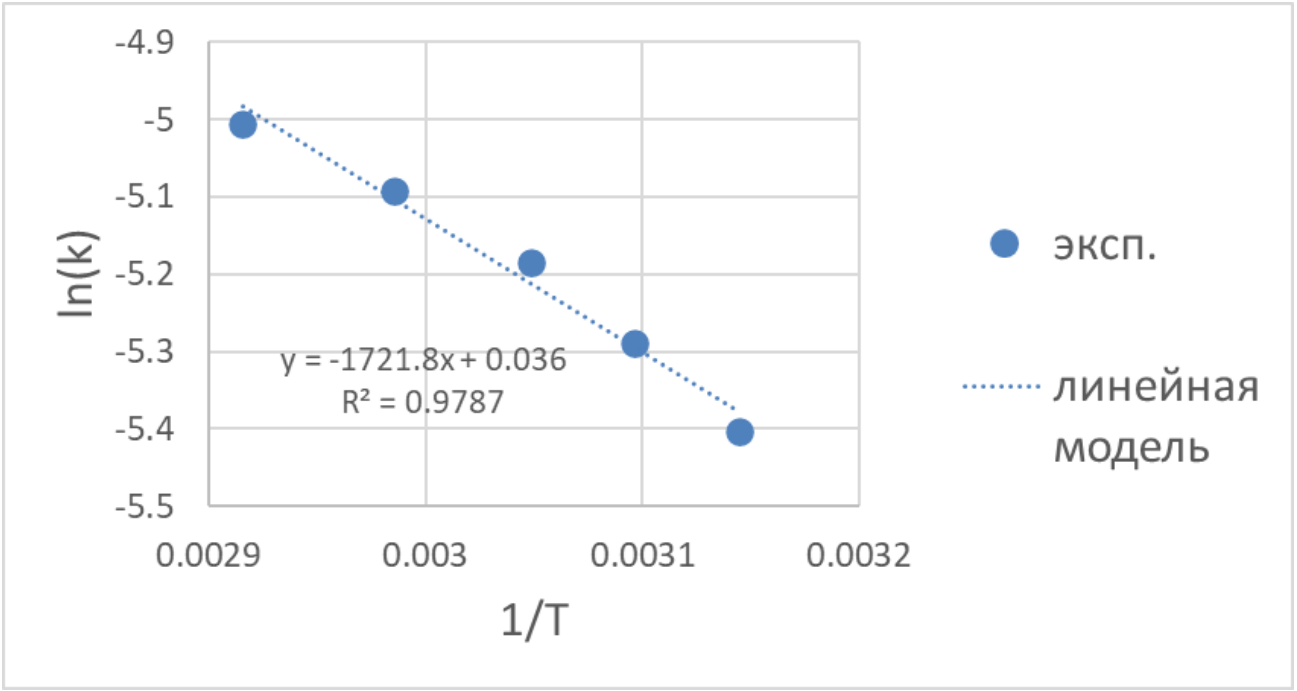


Простые эмпирические модели в Excel





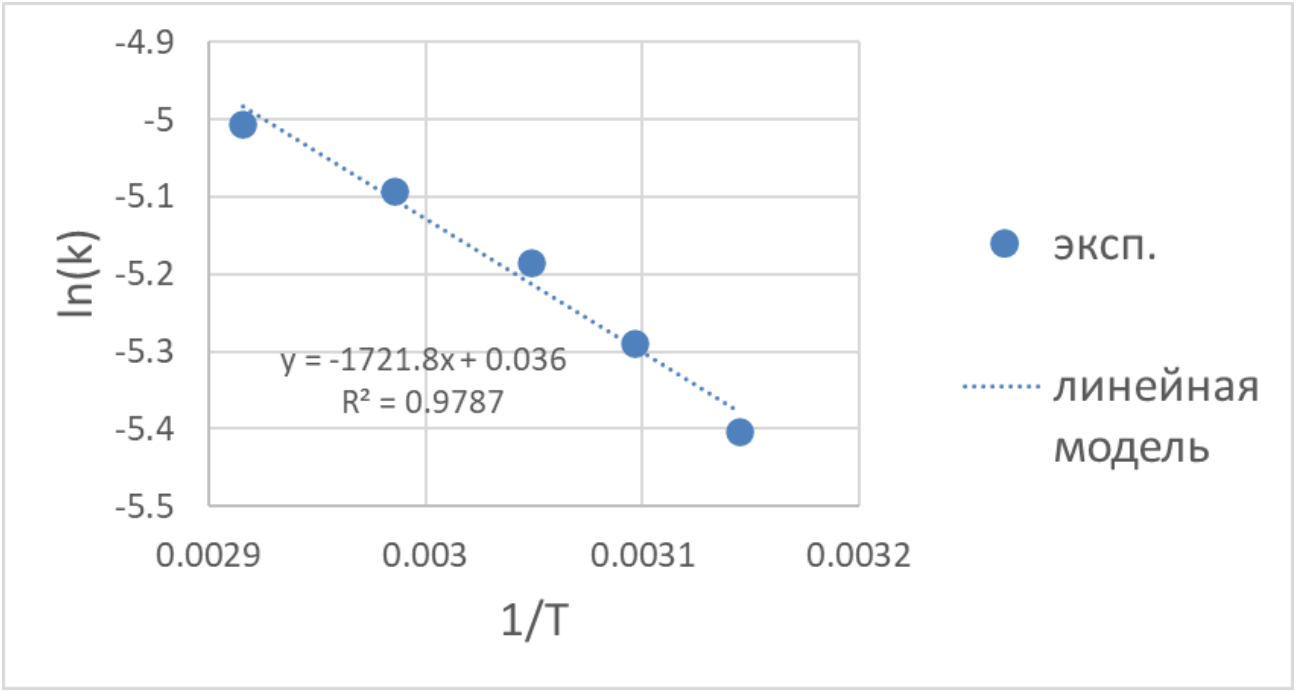
Простые эмпирические модели в Excel



$$y = -1721.8 \cdot x + 0.036$$

$$\ln(k) = -\frac{E}{R} \cdot \frac{1}{T} + \ln(\upsilon_0)$$

Простые эмпирические модели в Excel



$$y = -1721.8 \cdot x + 0.036$$

$$\ln(k) = -\frac{E}{R} \cdot \frac{1}{T} + \ln(v_0)$$

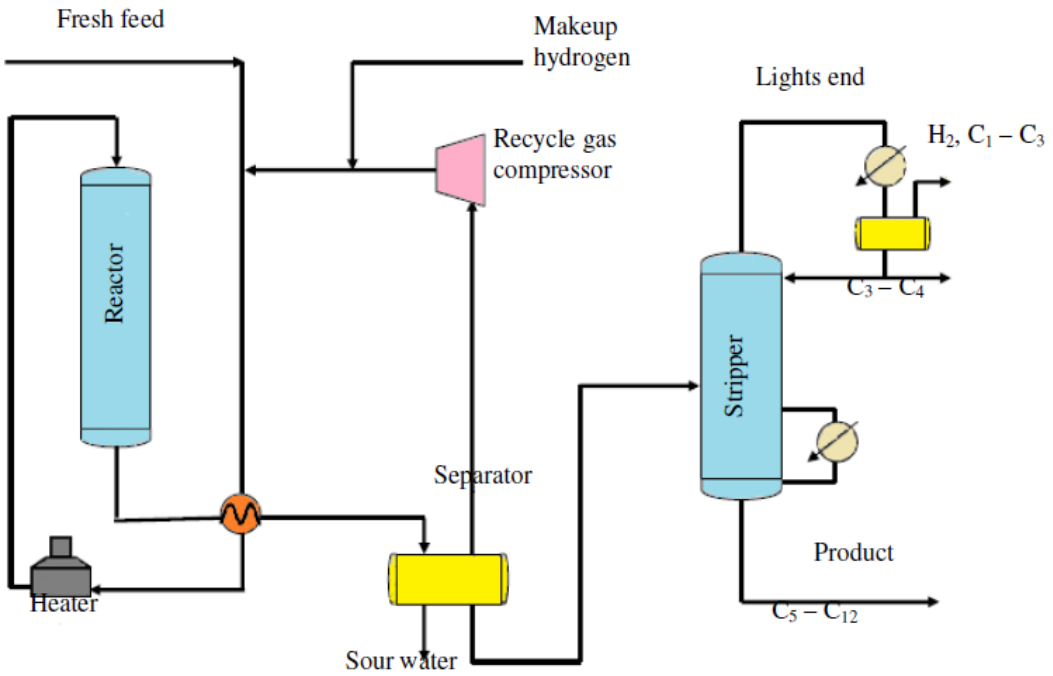
$$\ln(v_0) = 0.036$$

$$v_0 = e^{0.036} = 1.03$$

$$\frac{E}{R} = 1721.8$$

$$E = 1721.8 \cdot 8.31 = 14.3 \text{ кДж/моль}$$

Пример: гидроочистка бензиновой фракции





- Как скорость реакции гидроочистки бензиновой фракции зависит от давления ВСГ? Можно предположить, что скорость реакции описывается уравнением вида

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

Метод наименьших квадратов

✓ Необходимо определить параметры искомой зависимости θ_1, θ_2 таким образом, чтобы расчетная кривая лежала как можно ближе к экспериментальной кривой:

- ***Критерий близости:*** минимум суммы квадратов отклонений между экспериментальными и расчетными значениями

$$SSE = \sum_{i=1}^{n} \left(Y_i^{exp} - Y_i^{model}\right)^2 =$$

$$= \sum_{i=1}^{n} \left(Y_i^{exp} - \theta_1 \cdot P/(1 + \theta_2 \cdot P)\right)^2 \rightarrow min$$

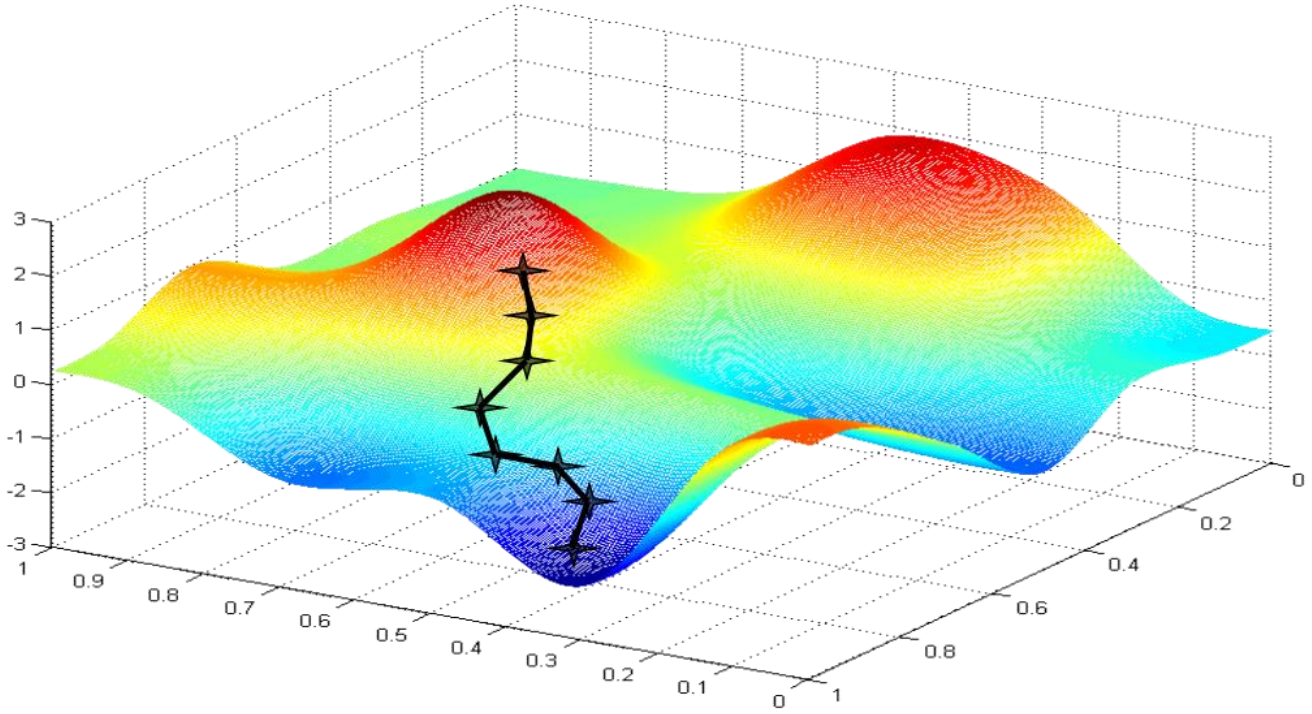
- ***Условие минимума SSE:*** равенство нулю производных по параметрам θ_1, θ_2

$$\begin{cases} \dfrac{\partial(SSE)}{\partial\theta_1} = 0 \\ \dfrac{\partial(SSE)}{\partial\theta_2} = 0 \end{cases}$$

Метод градиентного спуска

$$SSE = \sum_{i=1}^{n} \left(Y_i^{exp} - Y_i^{model}\right)^2 = \sum_{i=1}^{n} \left(Y_i^{exp} - \theta_1 \cdot P/(1 + \theta_2 \cdot P)\right)^2 \rightarrow min$$

- Начинаем с произвольных значений параметров (θ_1, θ_2)
- Продолжаем изменять θ_1, θ_2, чтобы уменьшить $SSE(\theta_1, \theta_2)$ до тех пор пока предположительно не достигнем минимума

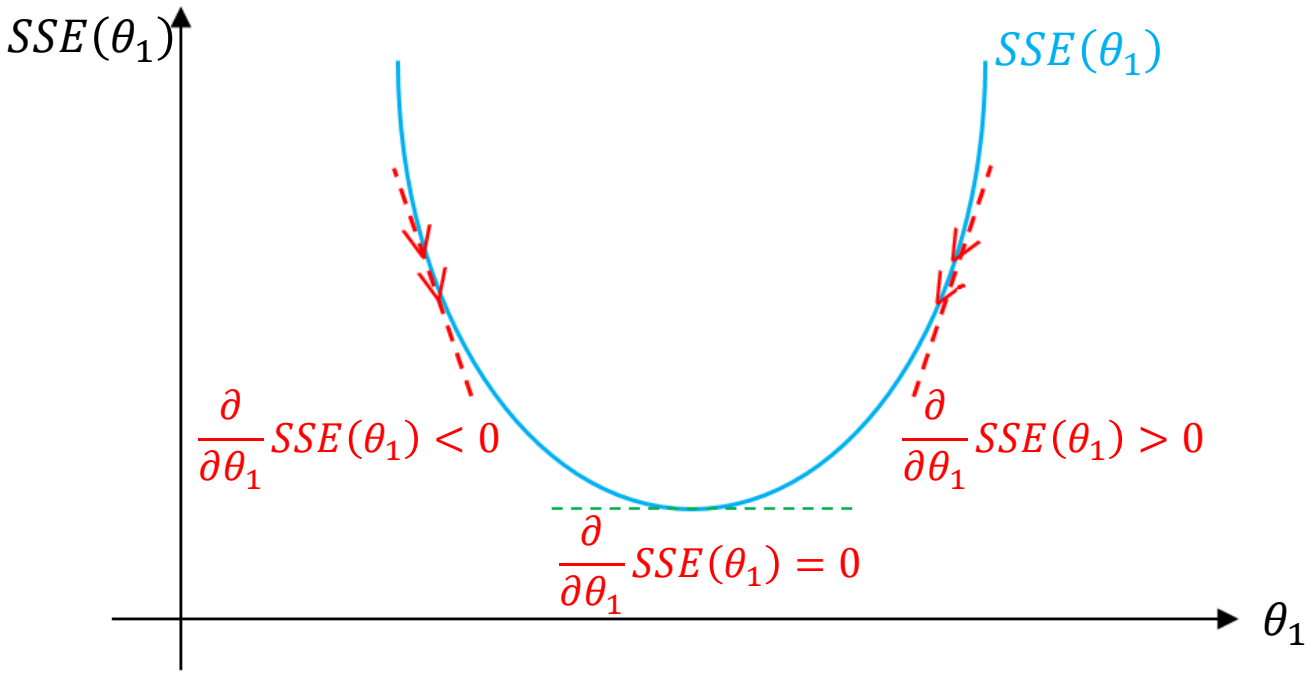


Метод градиентного спуска - алгоритм

$$SSE = \sum_{i=1}^{n} \left(Y_i^{exp} - Y_i^{model}\right)^2 = \sum_{i=1}^{n} \left(Y_i^{exp} - \theta_1 \cdot P/(1 + \theta_2 \cdot P)\right)^2 \rightarrow min$$

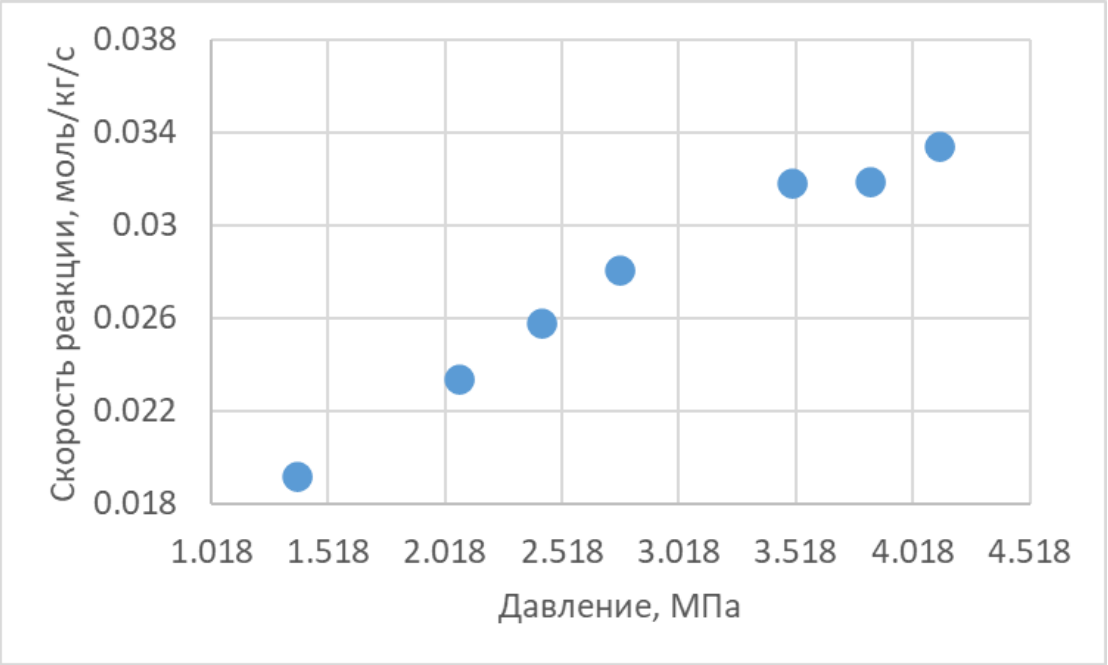
✓ Повторять $\theta_i := \theta_i - \alpha \cdot \frac{\partial}{\partial \theta_i} \cdot SSE(\theta_1, \theta_2)$ до достижения сходимости

(одновременно обновляются значения $i = 1$ и $i = 2$)



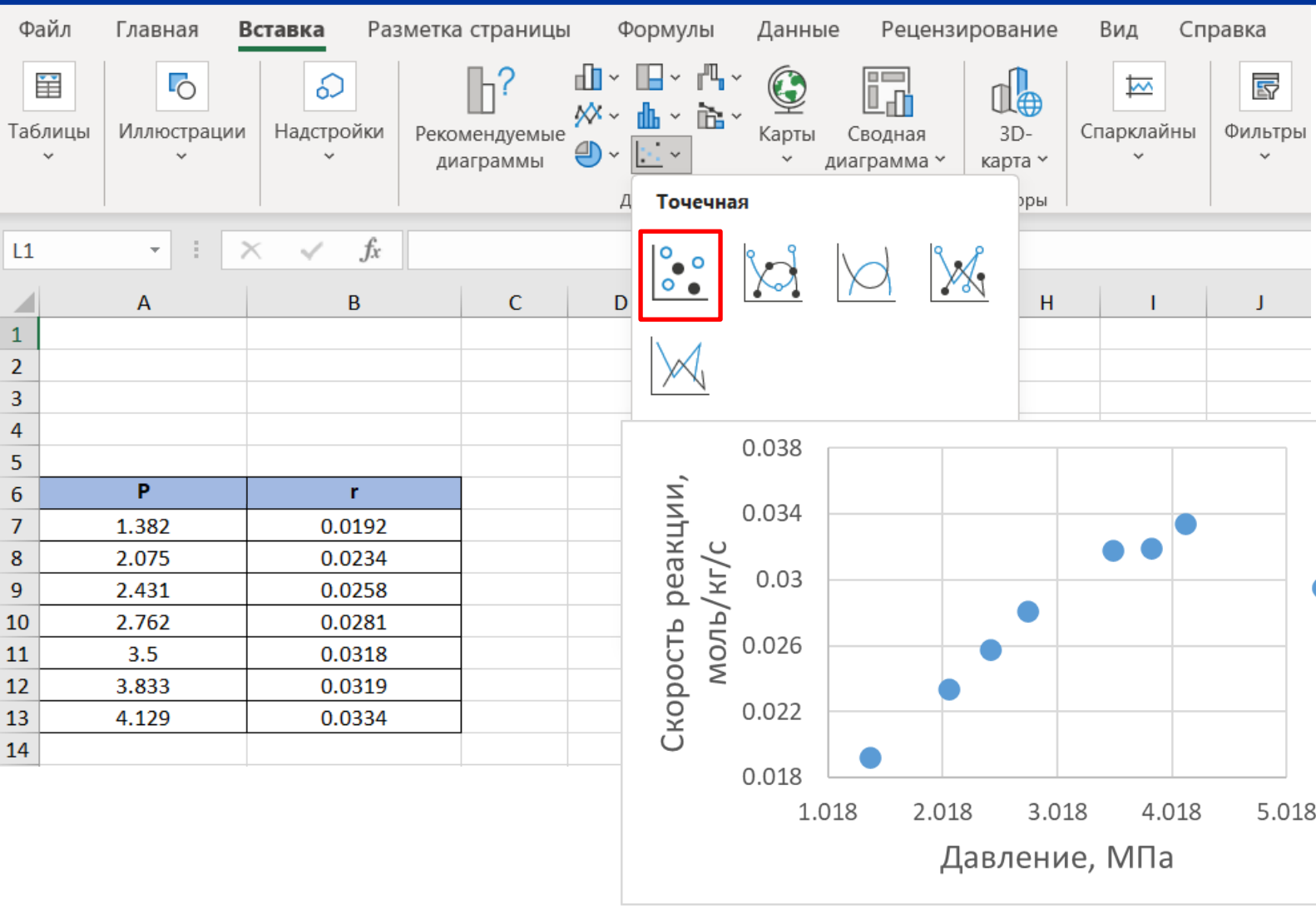
Пример: гидроочистка бензиновой фракции

r, моль/с/кг cat	P, МПа
0.0192	1.382
0.0234	2.075
0.0258	2.431
0.0281	2.762
0.0318	3.5
0.0319	3.833
0.0334	4.129

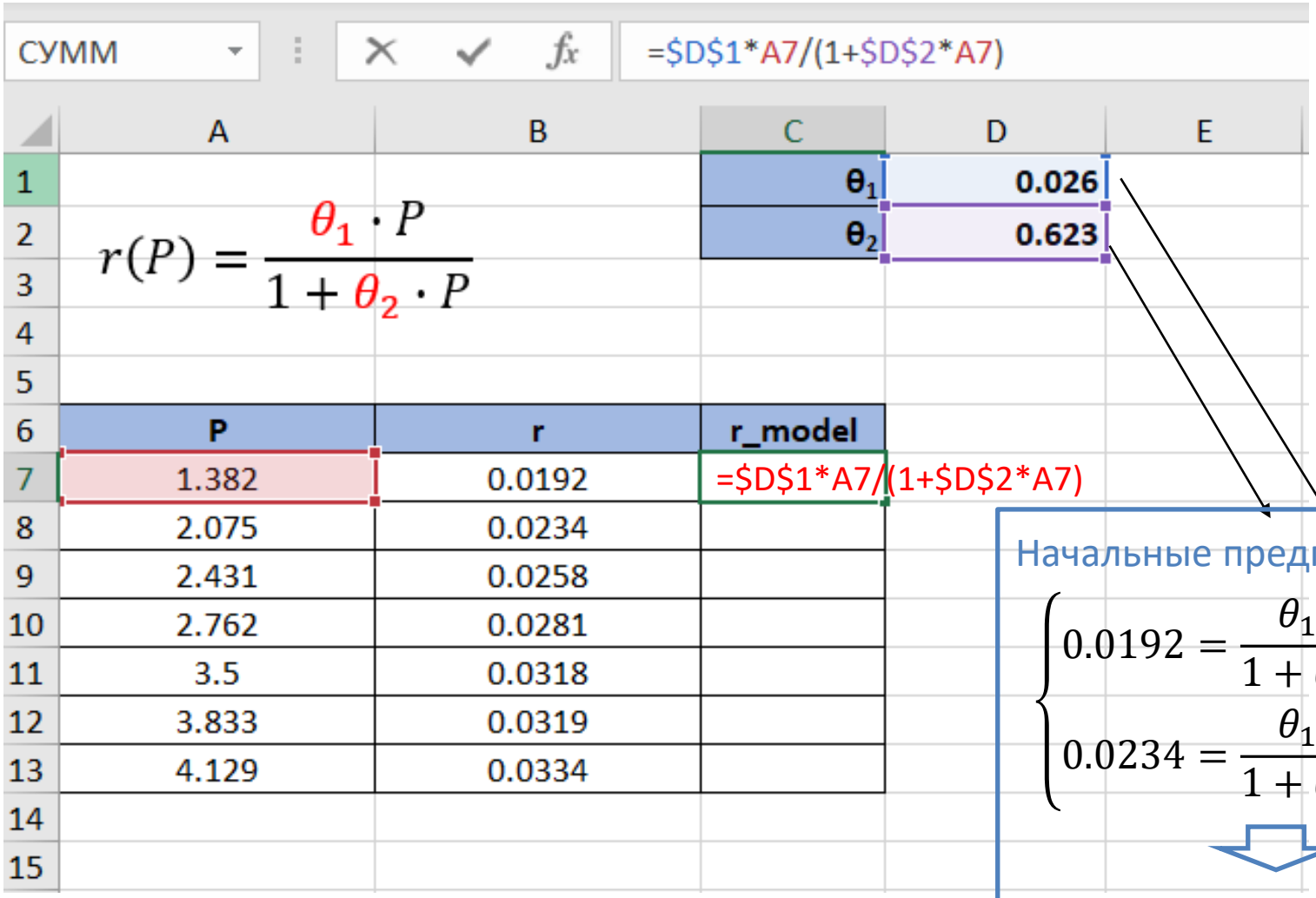


$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

Пример в Excel: гидроочистка бензиновой фракции



Пример в Excel: гидроочистка бензиновой фракции



Начальные предположения:

$$\begin{cases} 0.0192 = \dfrac{\theta_1 \cdot 1.382}{1 + \theta_2 \cdot 1.382} \\ 0.0234 = \dfrac{\theta_1 \cdot 2.075}{1 + \theta_2 \cdot 2.075} \end{cases}$$

$$\theta_1 \approx 0.026,\ \theta_2 \approx 0.623$$

Пример в Excel: гидроочистка бензиновой фракции

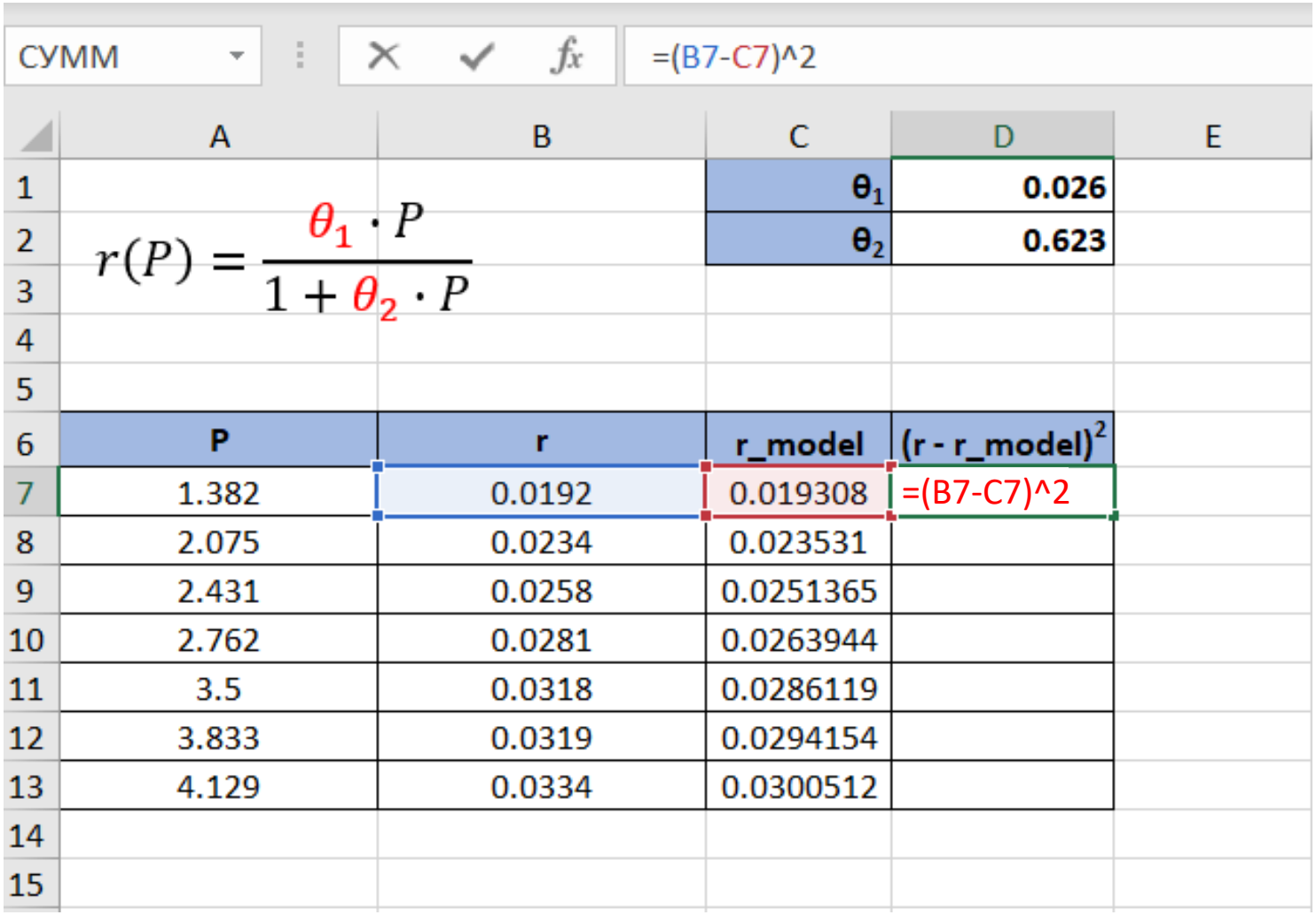
F18

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

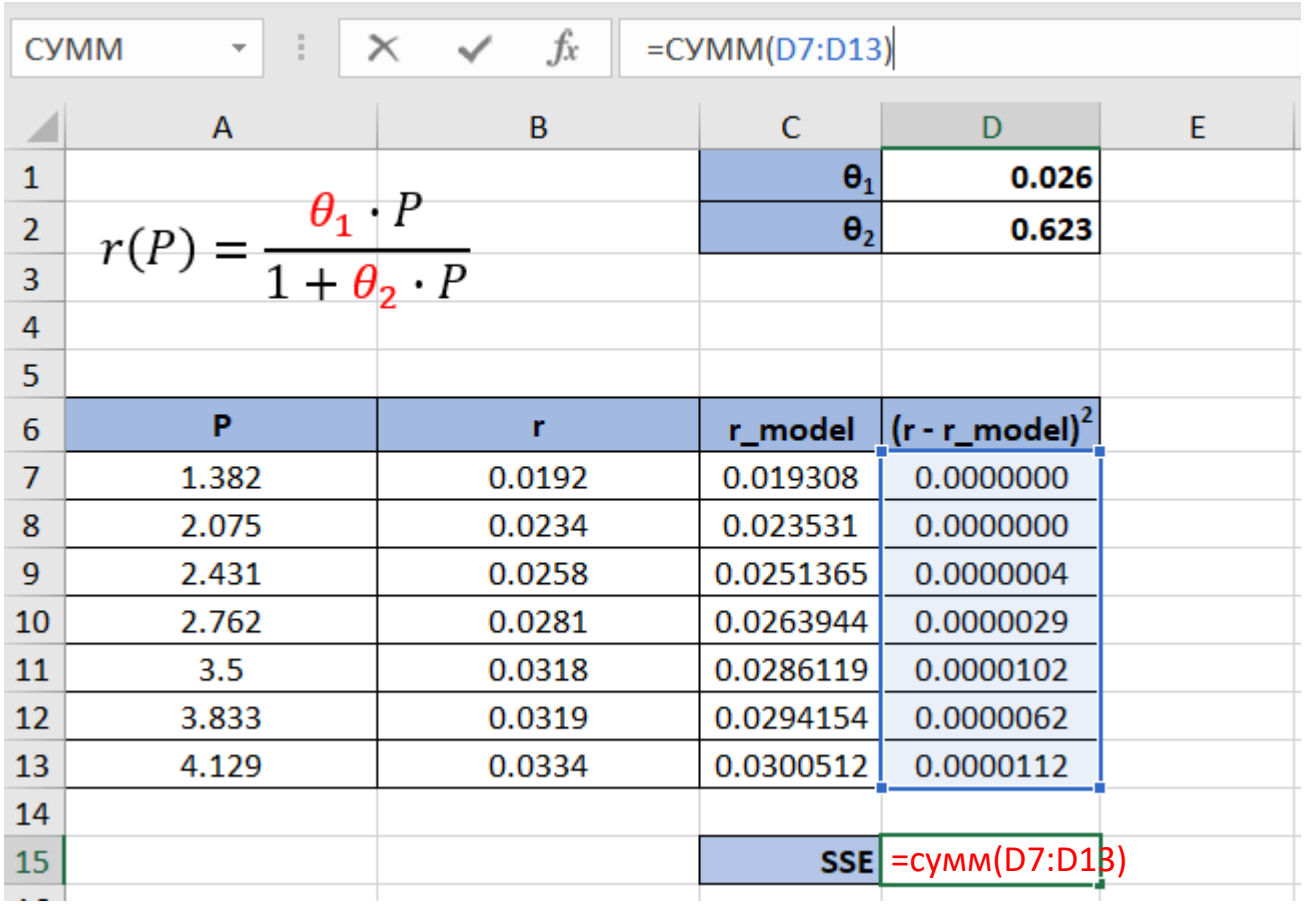
	C	D
1	θ_1	0.026
2	θ_2	0.623

P	r	r_model
1.382	0.0192	0.019308
2.075	0.0234	0.023531
2.431	0.0258	0.0251365
2.762	0.0281	0.0263944
3.5	0.0318	0.0286119
3.833	0.0319	0.0294154
4.129	0.0334	0.0300512

Пример в Excel: гидроочистка бензиновой фракции



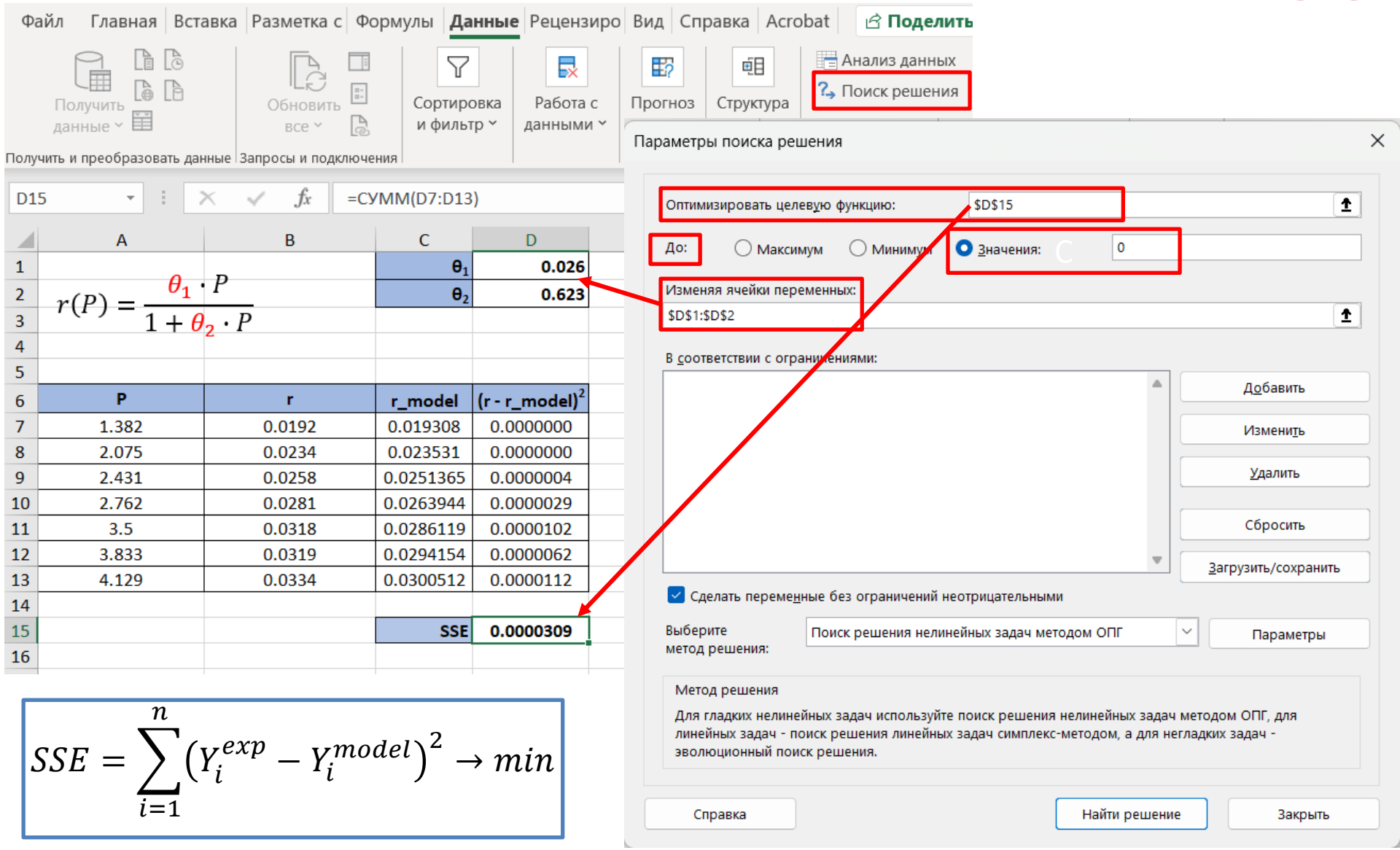
Пример в Excel: гидроочистка бензиновой фракции



Сумма квадратов отклонений или остаточная сумма:

$$SSE = \sum_{i=1}^{n} \left(Y_i^{exp} - Y_i^{model}\right)^2$$

Пример в Excel: гидроочистка бензиновой фракции



$$SSE = \sum_{i=1}^{n} \left(Y_i^{exp} - Y_i^{model}\right)^2 \rightarrow min$$

Пример в Excel: гидроочистка бензиновой фракции

D15 | =СУММ(D7:D13)

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

	A	B	C	D	E
1			θ_1	0.020513485	
2			θ_2	0.373192094	
3					
4					
5					
6	P	r	r_model	$(r - r_model)^2$	
7	1.382	0.0192	0.0187034	0.0000002	
8	2.075	0.0234	0.023989	0.0000003	
9	2.431	0.0258	0.026147	0.0000001	
10	2.762	0.0281	0.0279001	0.0000000	
11	3.5	0.0318	0.0311326	0.0000004	
12	3.833	0.0319	0.0323514	0.0000002	
13	4.129	0.0334	0.0333346	0.0000000	
14					
15			SSE	0.0000014	→ 0
16					

$$r(P) = \frac{0.0205 \cdot P}{1 + 0.3732 \cdot P}$$

Пример в Excel: гидроочистка бензиновой фракции

O28

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

	A	B	C	D	E	F
1			θ_1	0.020513485		
2			θ_2	0.373192094		
3						
4						
5						
6	P	r	r_model	$(r - r_model)^2$	$(r - r_mean)^2$	r - r_model
7	1.382	0.0192	0.0187034	0.0000002	5.6667E-05	0.000496646
8	2.075	0.0234	0.023989	0.0000003	1.1074E-05	-0.000589018
9	2.431	0.0258	0.026147	0.0000001	8.6076E-07	-0.000346968
10	2.762	0.0281	0.0279001	0.0000000	1.883E-06	0.000199933
11	3.5	0.0318	0.0311326	0.0000004	2.5728E-05	0.000667376
12	3.833	0.0319	0.0323514	0.0000002	2.6752E-05	-0.00045135
13	4.129	0.0334	0.0333346	0.0000000	4.4519E-05	6.54178E-05
14	r_mean	0.026727771		Σ	0.00016748	
15			SSE	0.0000014		
16			R^2	0.991597088		
17						

B14 = СРЗНАЧ(B7:B13)

E7 = (B7-B14)^2

E14 = СУММ(E7:E13)

D16 = 1-D15/E14

F7 = B7-C7

$$R^2 = 1 - \frac{SSE}{TSS} = 1 - \frac{\sum_{i=1}^{n}\left(y_i^{exp} - y_i^{model}\right)^2}{\sum_{i=1}^{n}\left(y_i^{exp} - \overline{y^{exp}}\right)^2}$$

- остаточная сумма

- полная дисперсия

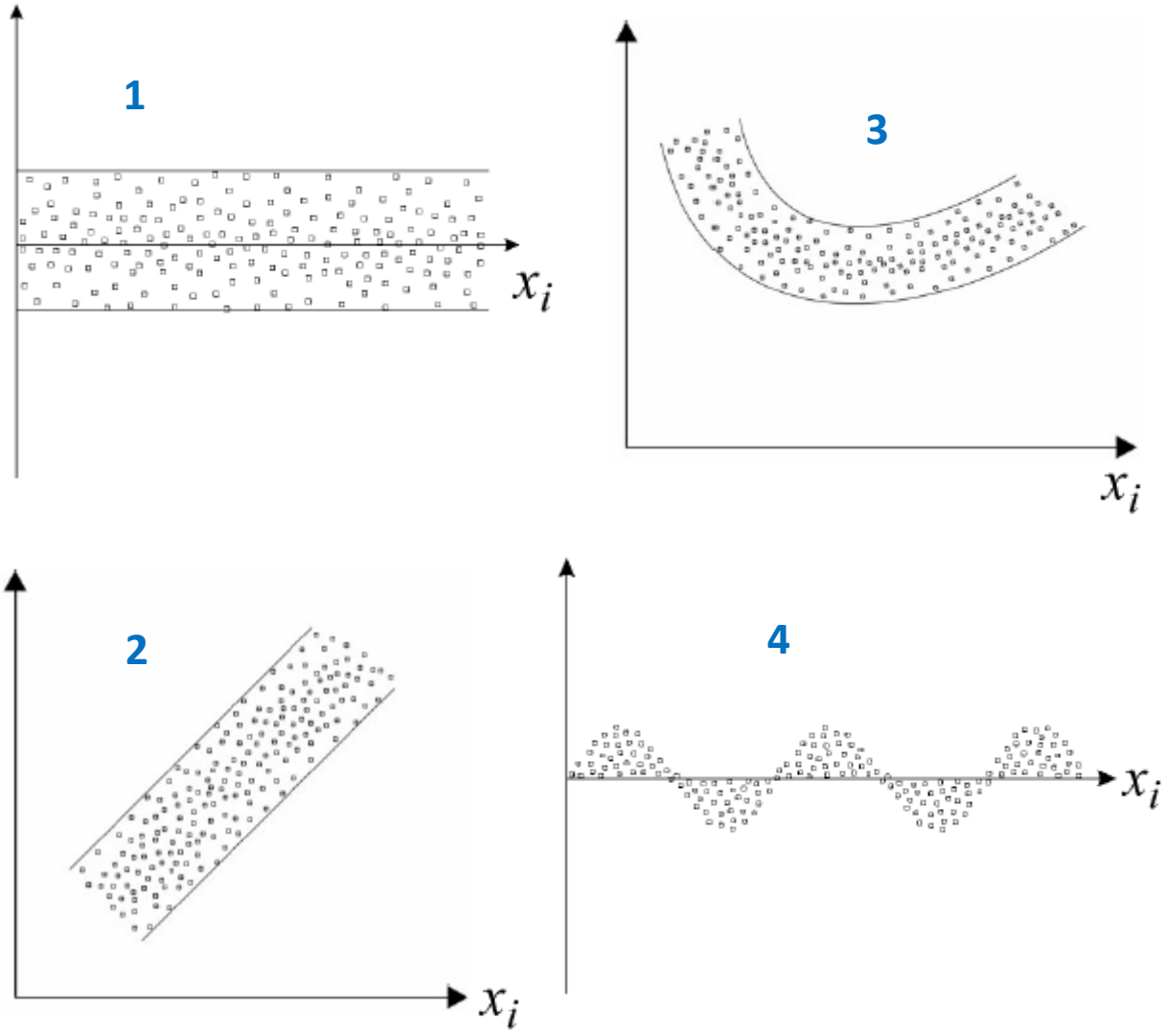
Пример в Excel: гидроочистка бензиновой фракции

P	r	r_model
1.382	0.0192	0.018052
2.075	0.0234	0.0235254
2.431	0.0258	0.0258109
2.762	0.0281	0.0276899
3.5	0.0318	0.0312083
3.833	0.0319	0.0325532
4.129	0.0334	0.0336458

$$r(P) = \frac{0.0205 \cdot P}{1 + 0.3732 \cdot P}$$



График остатков

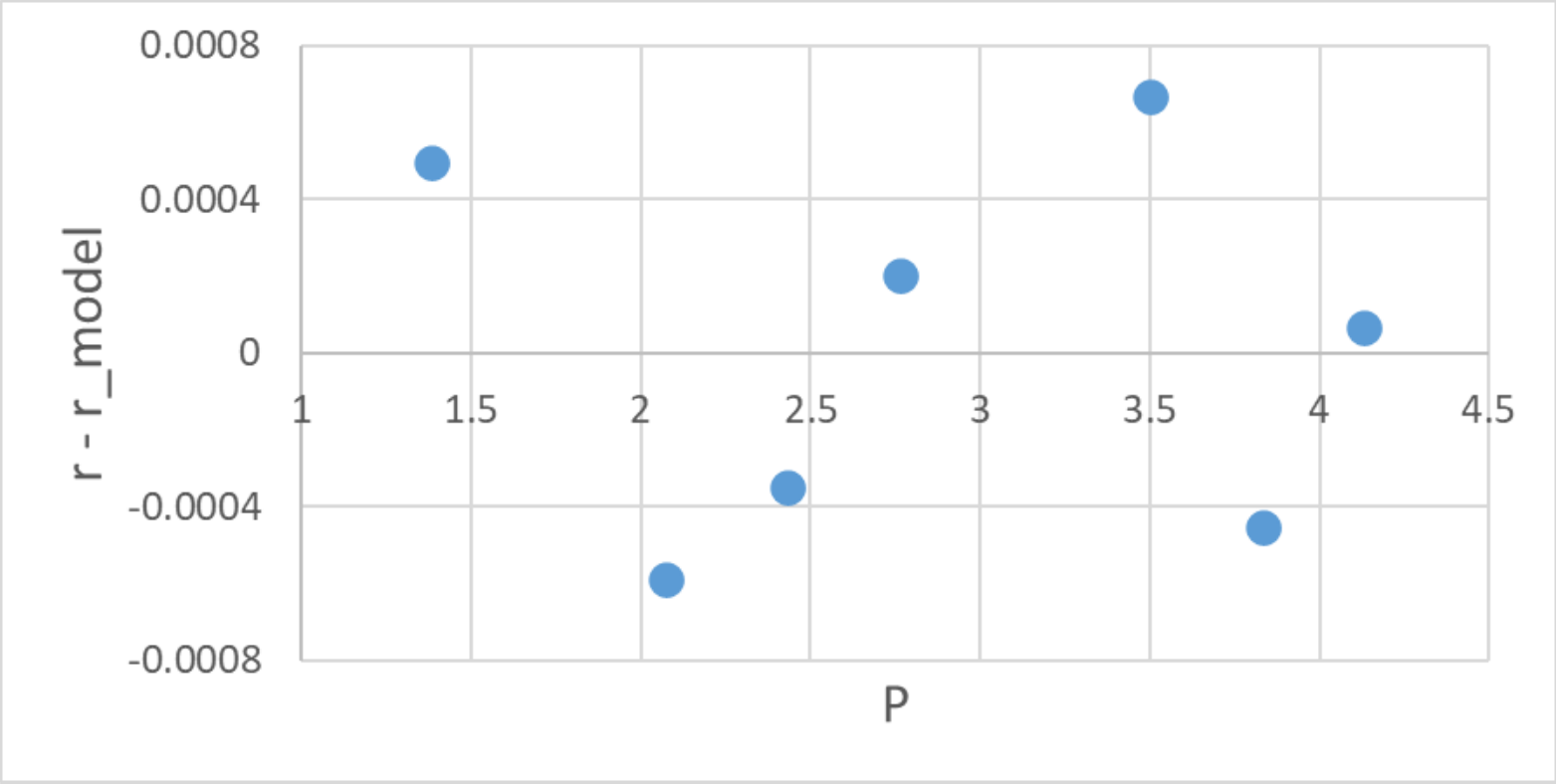


$$\text{Остаток} = y_i^{exp} - y_i^{model}$$

1. Полностью случайное распределение ошибок
2. Нужно добавить линейную часть
3. Нужно добавить квадратичную часть
4. Нужно добавить синусоидальную часть

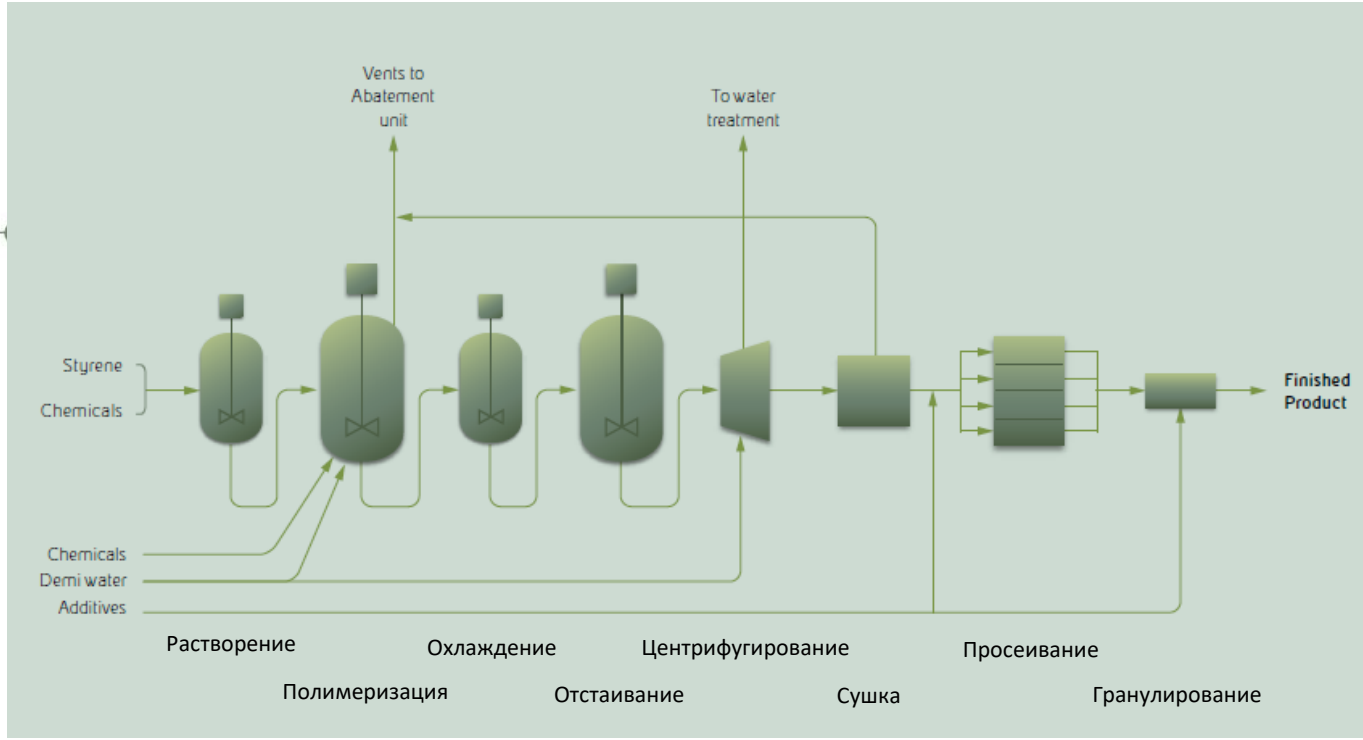
Пример в Excel: гидроочистка бензиновой фракции

График остатков



Пример: производство полистирола







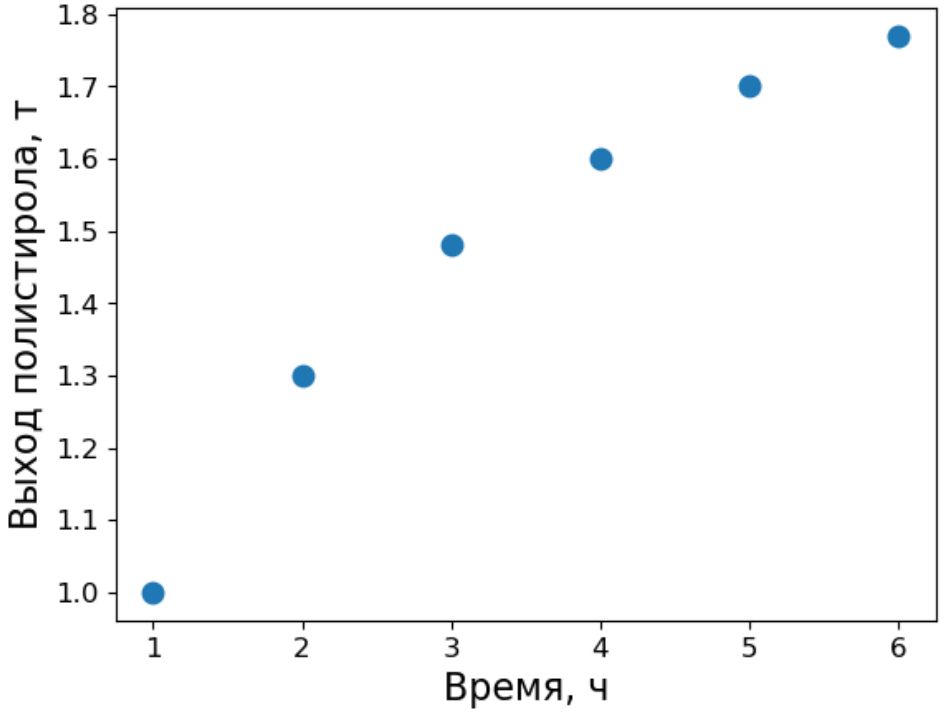


Пример: производство полистирола

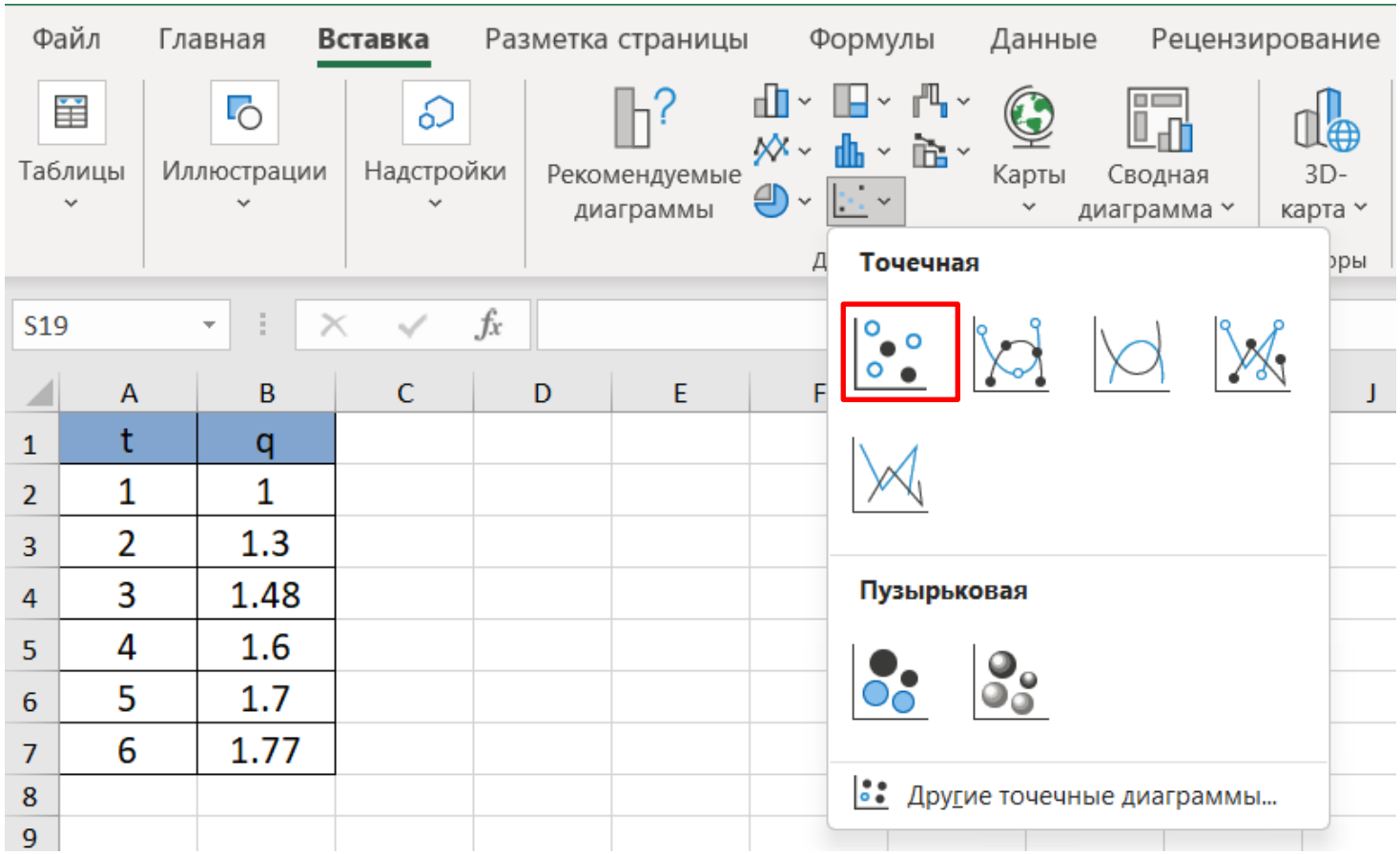


- Данные работы установки суспензионной полимеризации стирола – зависимость количества полистирола (q в тоннах) от времени (t в часах):

t, ч	q, т
1	1
2	1.3
3	1.48
4	1.6
5	1.7
6	1.77



Производство ПС: полиномиальная регрессия в Excel

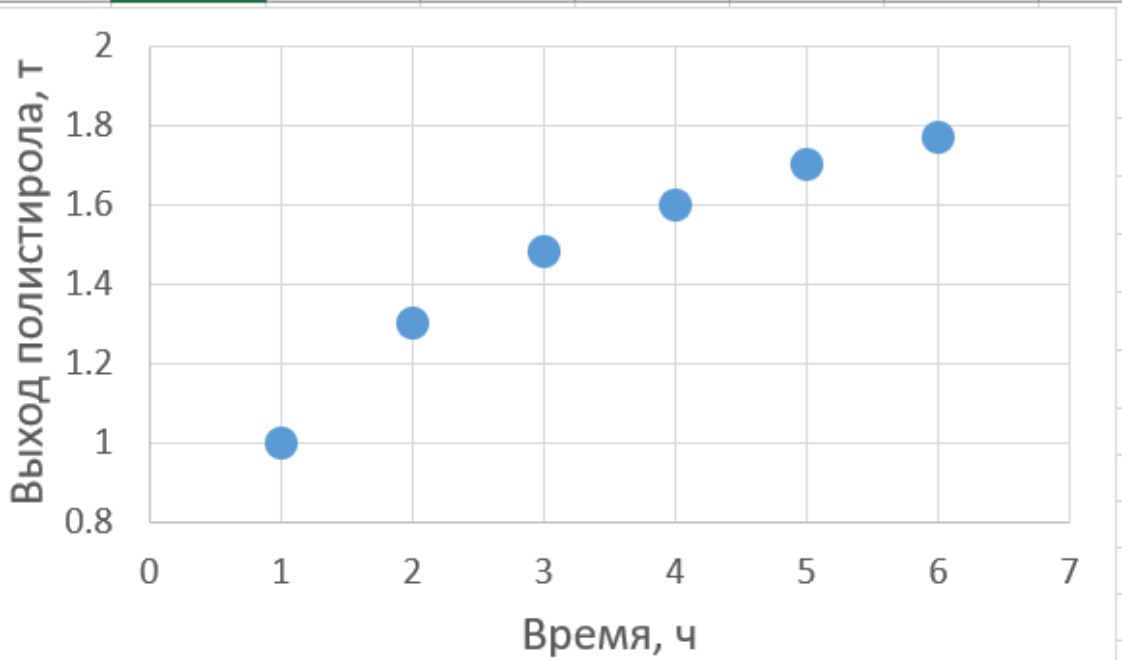


	A	B
1	t	q
2	1	1
3	2	1.3
4	3	1.48
5	4	1.6
6	5	1.7
7	6	1.77

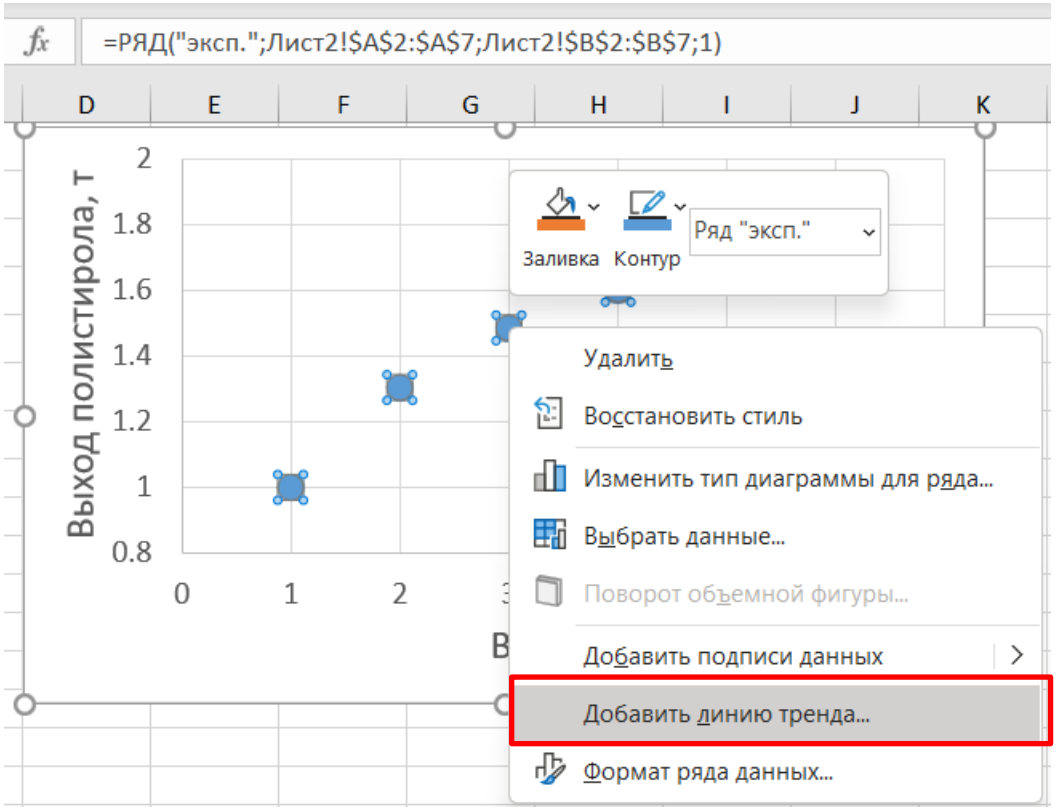
Производство ПС: полиномиальная регрессия в Excel

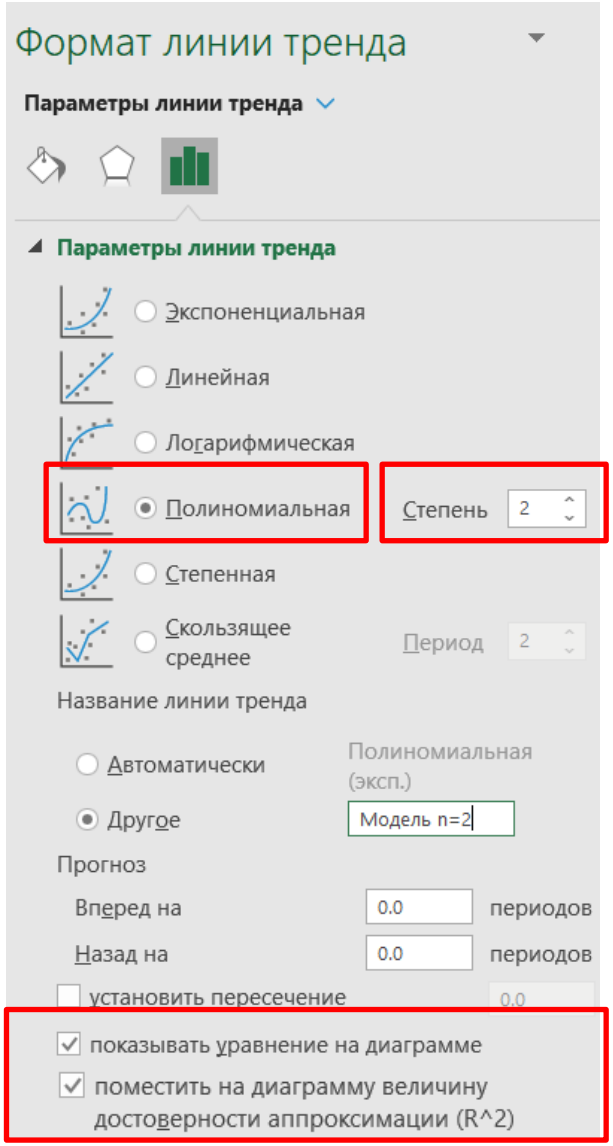
E22

	A	B	C	D	E	F	G	H	I	J	K
1	t	q									
2	1	1									
3	2	1.3									
4	3	1.48									
5	4	1.6									
6	5	1.7									
7	6	1.77									
8											
9											
10											
11											
12											
13											
14											



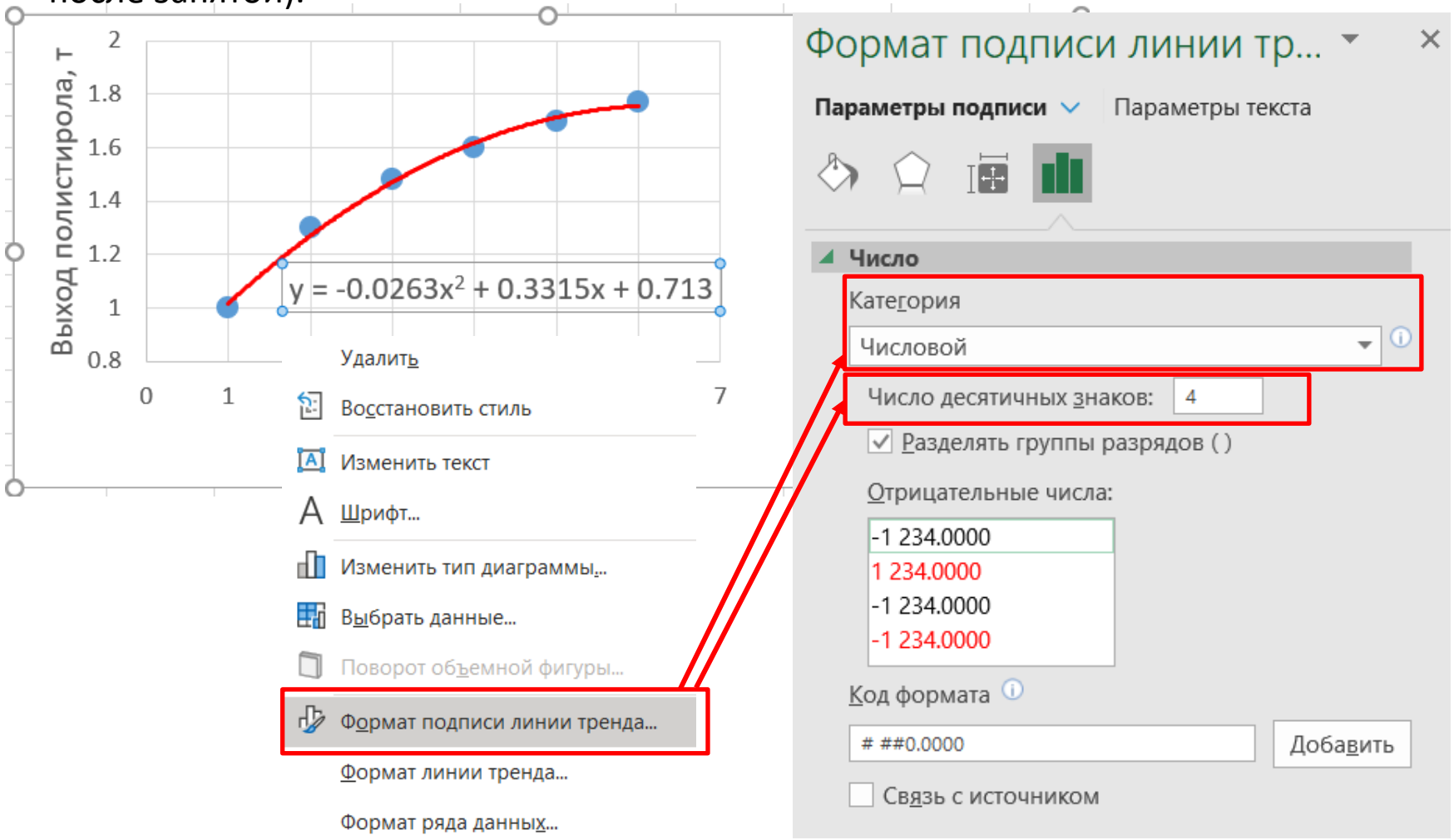
Производство ПС: полиномиальная регрессия в Excel



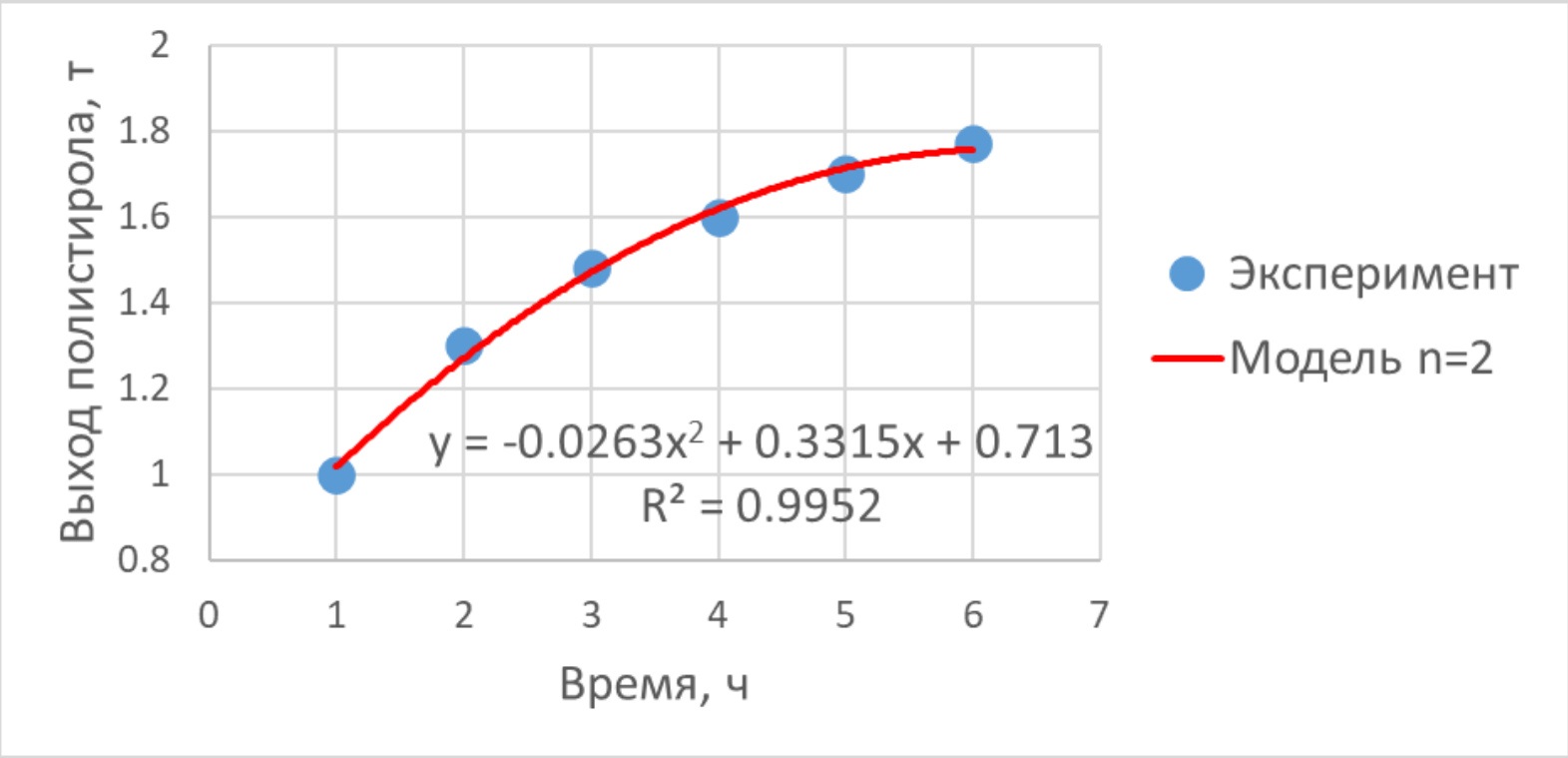


Производство ПС: полиномиальная регрессия в Excel

- В некоторых случаях может потребоваться изменить формат отображаемого уравнения для более точного отображения коэффициентов (например, формат и/или число знаков после запятой).

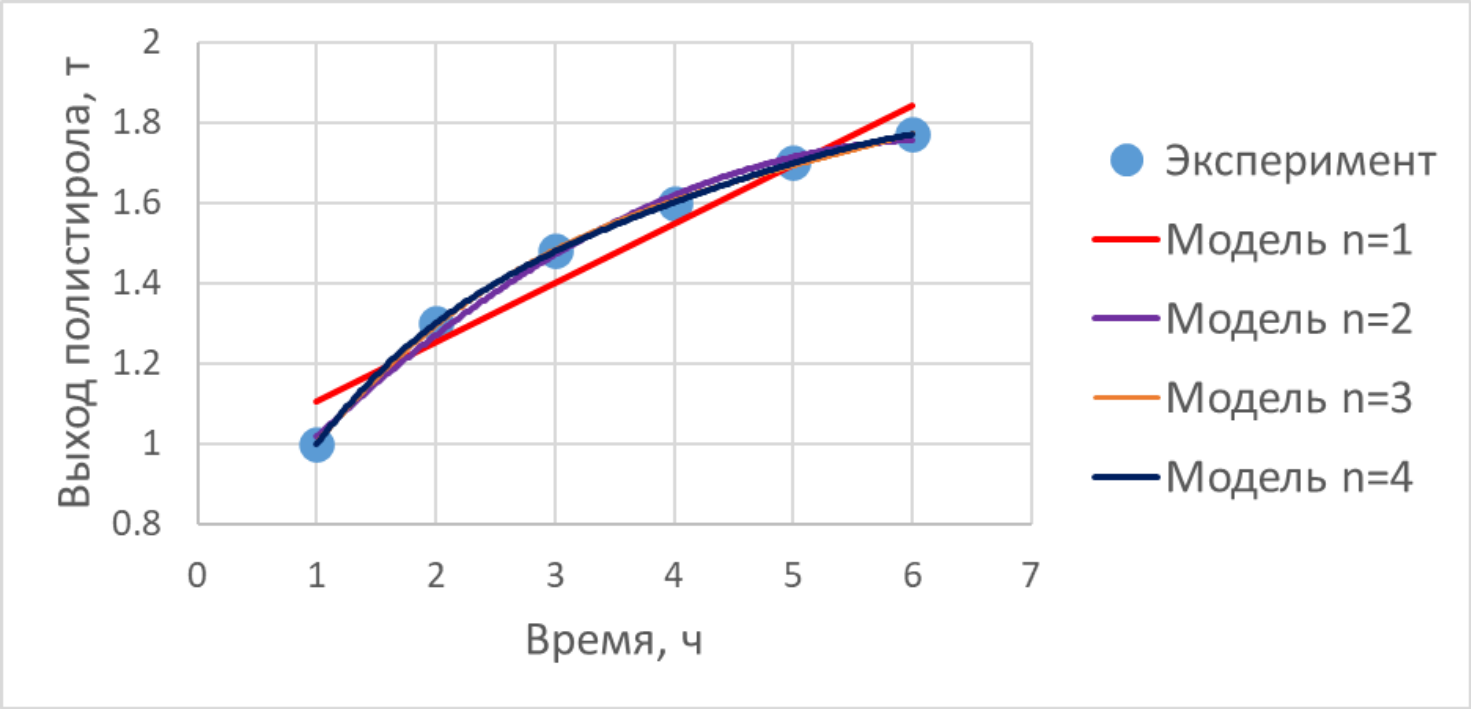


Производство ПС: полиномиальная регрессия в Excel



$$q = -0.0263 \cdot t^2 + 0.3315 \cdot t + 0.713$$

Производство ПС: полиномиальная регрессия в Excel



$$q = 0.1477 \cdot t + 0.958$$

$$q = -0.0263 \cdot t^2 + 0.3315 \cdot t + 0.713$$

$$q = 0.0053 \cdot t^3 - 0.0817 \cdot t^2 + 0.4988 \cdot t + 0.58$$

$$q = -0.0015 \cdot t^4 + 0.0257 \cdot t^3 - 0.179 \cdot t^2 + 0.6796 \cdot t + 0.475$$

Общий случай - вид эмпирической зависимости

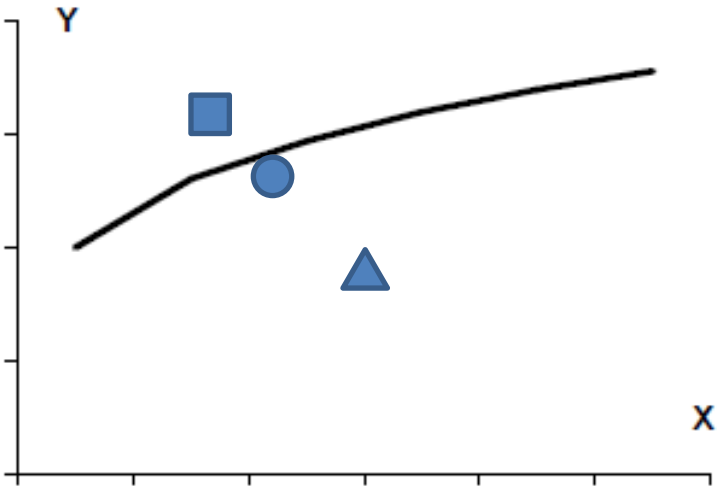
✓ В общем случае необходимо подобрать зависимость, которая могла бы описать экспериментальные данные. ***Для монотонных зависимостей:***

№	Формула	X_k	Y_k	Приведение к линейному виду
1	$Y = a \cdot X^b$	$\sqrt{X_1 \cdot X_n}$	$\sqrt{Y_1 \cdot Y_n}$	$U=A+bZ$; $U=\lg Y$; $A=\lg a$; $Z=\lg X$
2	$Y = a \cdot b^X$	$(X_1+X_n)/2$	$\sqrt{Y_1 \cdot Y_n}$	$U=A+BX$; $U=\lg Y$; $A=\lg a$; $B=\lg b$
3	$Y=\frac{1}{a+b \cdot X}$	$(X_1+X_n)/2$	$\frac{2 \cdot Y_1 \cdot Y_n}{Y_1+Y_n}$	$U=a+bX$; $U=1/Y$
4	$Y=a+b \cdot \lg X$	$\sqrt{X_1 \cdot X_n}$	$(Y_1+Y_n)/2$	$Y=a+bZ$; $Z=\lg X$
5	$Y = a + \frac{b}{X}$	$\frac{2 \cdot X_1 \cdot X_n}{X_1+X_n}$	$(Y_1+Y_n)/2$	$Y=a+bZ$; $Z=1/X$
6	$Y = \frac{a \cdot X}{b+X}$	$\frac{2 \cdot X_1 \cdot X_n}{X_1+X_n}$	$\frac{2 \cdot Y_1 \cdot Y_n}{Y_1+Y_n}$	$U=A+BZ$; $U=1/Y$; $Z=1/X$; $A=1/a$; $B=b/a$

Вид эмпирической зависимости

✓ Выбор вида зависимости методом средних точек (для равномерно непрерывной зависимости в исследуемом интервале):

- Расчет координат средних точек X_k и Y_k;
- Нанесение средних точек на экспериментальную зависимость;
- Выбор функционального вида зависимости, средняя точка которого лежит ближе всего к экспериментальной кривой.



$Y = a + b \cdot X$

$Y = a \cdot X^b$

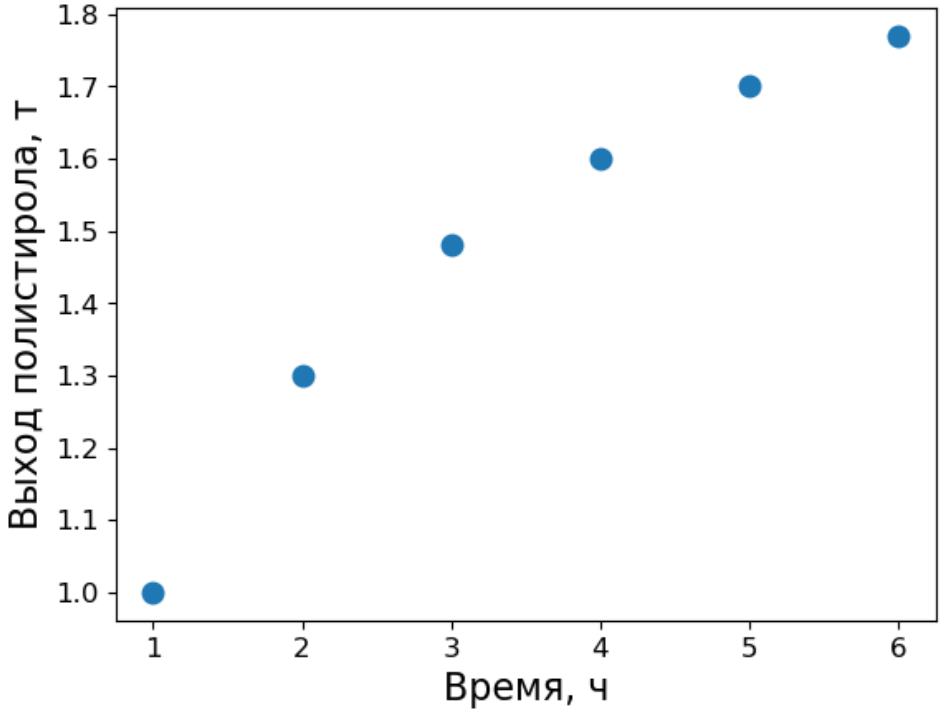
$Y = \frac{a \cdot X}{b + X}$

Пример: производство полистирола



- Данные работы установки суспензионной полимеризации стирола – зависимость количества полистирола (q в тоннах) от времени (t в часах):

t, ч	q, т
1	1
2	1.3
3	1.48
4	1.6
5	1.7
6	1.77



Производство ПС: метод средних точек

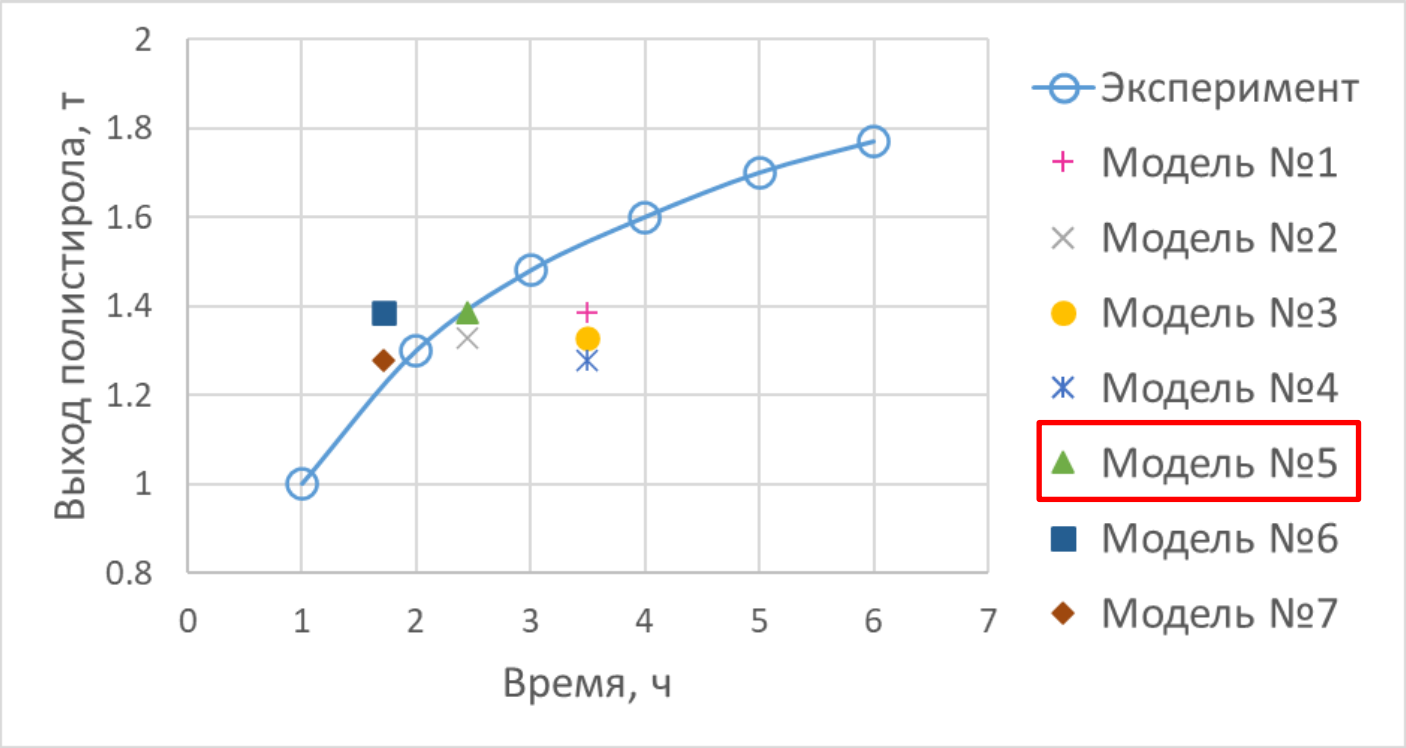
M15

	A	B	C	D	E	F	G	H
1	t	q					t_k	q_k
2	1	1		Модель №1	$q = a + b \cdot t$		3.5	1.385
3	2	1.3		Модель №2	$q = a \cdot t^b$		2.449	1.33
4	3	1.48		Модель №3	$q = a \cdot b^t$		3.5	1.33
5	4	1.6		Модель №4	$q = \frac{1}{a + b \cdot t}$		3.5	1.278
6	5	1.7		Модель №5	$q = a + b \cdot lgt$		2.449	1.385
7	6	1.77		Модель №6	$q = a + \frac{b}{t}$		1.714	1.385
8				Модель №7	$q = \frac{a \cdot t}{b + t}$		1.714	1.278
9								

G2 = (A2+A7)/2
G3 = КОРЕНЬ(A2*A7)
G4 = (A2+A7)/2
G5 = (A2+A7)/2
G6 = КОРЕНЬ(A2*A7)
G7 = 2*A2*A7/(A2+A7)
G8 = 2*A2*A7/(A2+A7)

H2 = (B2+B7)/2
H3 = КОРЕНЬ(B2*B7)
H4 = КОРЕНЬ(B2*B7)
H5 = 2*B2*B7/(B2+B7)
H6 = (B2+B7)/2
H7 = (B2+B7)/2
H8 = 2*B2*B7/(B2+B7)

Производство ПС: метод средних точек



$$q = a + b \cdot lgt$$

a - ?
b - ?

Производство ПС: метод средних точек

K16

	A	B	C	D	E	F	G	H	I	J	K
1	t	q					t_k	q_k		q_model	$(q - q_model)^2$
2	1	1		Модель №1	$q = a + b \cdot t$		3.5	1.385		1	0
3	2	1.3		Модель №2	$q = a \cdot t^b$		2.449	1.33		1.30103	1.06089E-06
4	3	1.48		Модель №3	$q = a \cdot b^t$		3.5	1.33		1.47712	8.28717E-06
5	4	1.6		Модель №4	$q = \frac{1}{a + b \cdot t}$		3.5	1.278		1.60206	4.24356E-06
6	5	1.7		Модель №5	$q = a + b \cdot lgt$		2.449	1.385		1.69897	1.06089E-06
7	6	1.77		Модель №6	$q = a + \frac{b}{t}$		1.714	1.385		1.77815	6.64429E-05
8				Модель №7	$q = \frac{a \cdot t}{b + t}$		1.714	1.278		SSE	8.10954E-05
9											
10									a	1	
11									b	1	

Начальные предположения:

при $t = 1$: $q = 1$

$q = a + b \cdot lgt$

$a \approx 1$

при $t = 2$: $q = 1.3$

$q = a + b \cdot lgt,\ a \approx 1$

$b \approx 1$

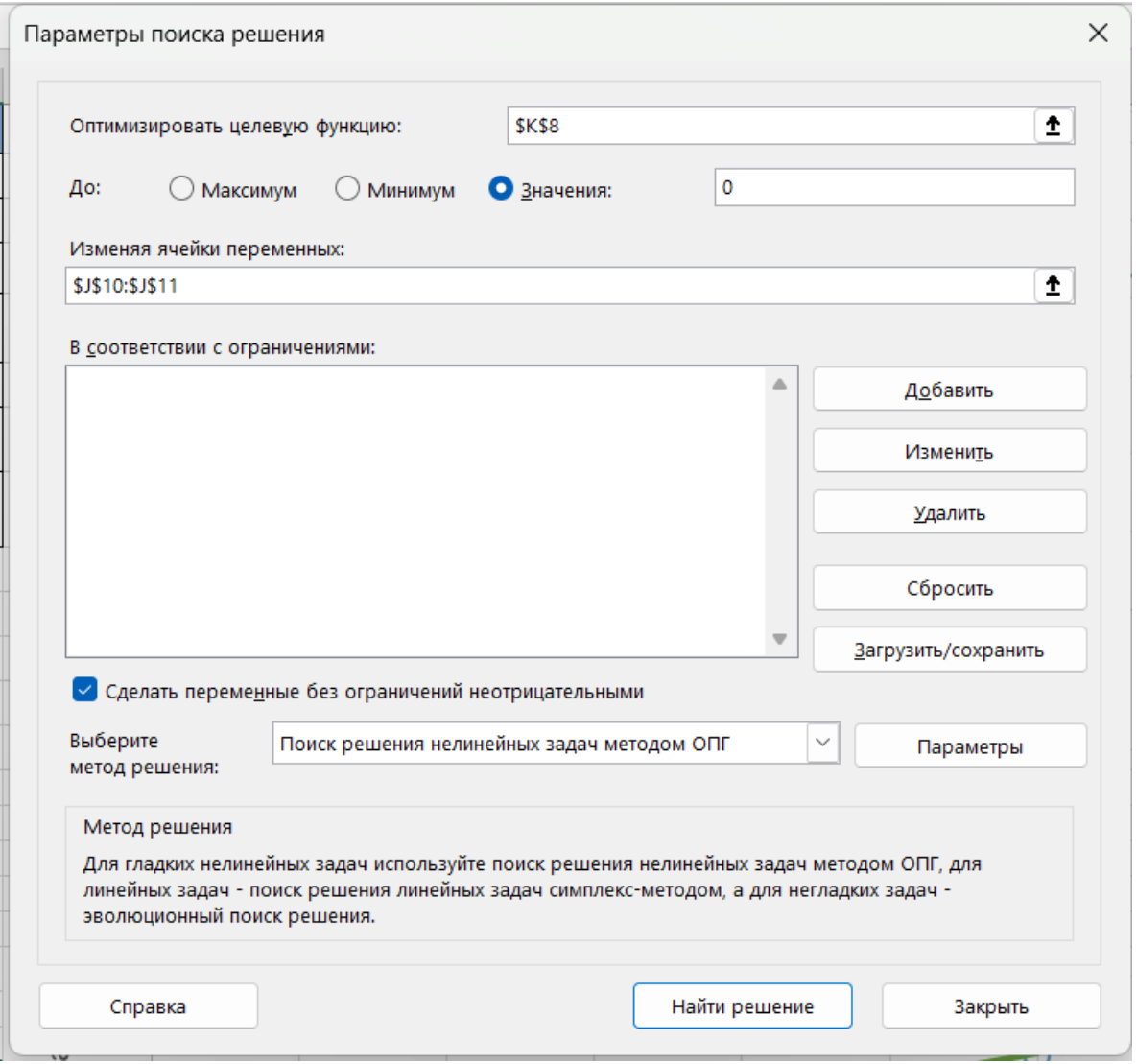
J2 = J10+J11*LOG10(A2)

K2 = (B2-J2)^2

K8 = СУММ(K2:K7)

Производство ПС: метод средних точек

I	J	K
	q_model	$(q - q_{model})^2$
	1	0
	1.30103	1.06089E-06
	1.47712	8.28717E-06
	1.60206	4.24356E-06
	1.69897	1.06089E-06
	1.77815	6.64429E-05
	SSE	8.10954E-05
a	1	
b	1	



Производство ПС: метод средних точек

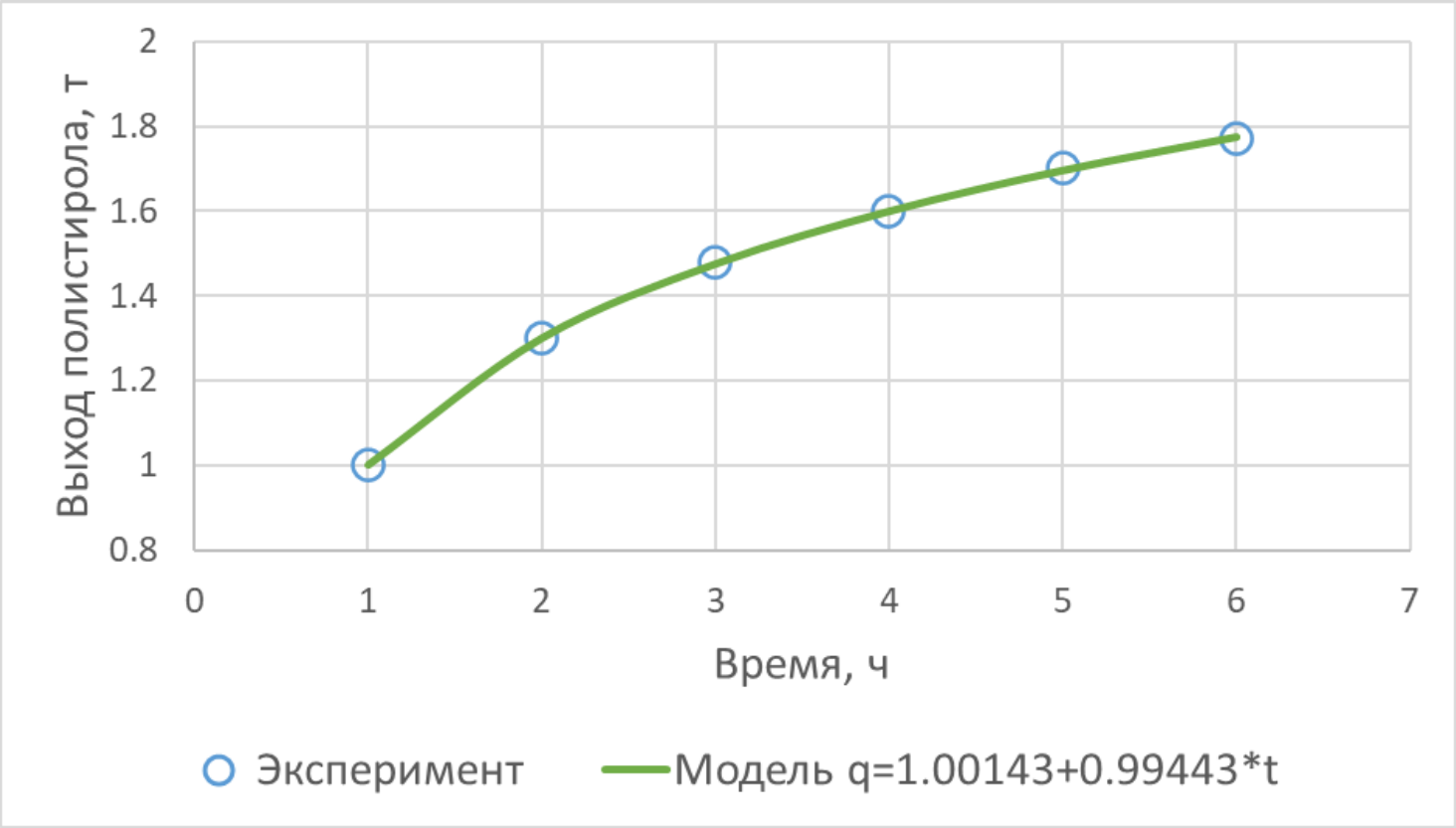
F24

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	t	q					t_k	q_k		q_model	$(q - q_model)^2$		$(q - q_mean)^2$
2	1	1		Модель №1	$q = a + b \cdot t$		3.5	1.385		1.00143	2.05073E-06		0.177032856
3	2	1.3		Модель №2	$q = a \cdot t^b$		2.449	1.33		1.30078	6.15139E-07		0.014581221
4	3	1.48		Модель №3	$q = a \cdot b^t$		3.5	1.33		1.47589	1.68579E-05		0.00351024
5	4	1.6		Модель №4	$q = \frac{1}{a + b \cdot t}$		3.5	1.278		1.60014	1.86537E-08		0.032129586
6	5	1.7		Модель №5	$q = a + b \cdot lgt$		2.449	1.385		1.69651	1.22046E-05		0.077979041
7	6	1.77		Модель №6	$q = a + \frac{b}{t}$		1.714	1.385		1.77525	2.7525E-05		0.121973659
8	q_mean	**1.421**		Модель №7	$q = \frac{a \cdot t}{b + t}$		1.714	1.278		SSE	**5.92721E-05**	Σ	**0.427206601**
9													
10									a	**1.00143**			
11									b	**0.99443**			
12													
13										R^2	**0.999861257**		

B8 = СРЗНАЧ(B2:B7)
M2 = (B2-B8)^2
M8 = СУММ(L2:L7)
K13 = 1-K8/M8

Производство ПС: метод средних точек

$$q = 1.00143 + 0.99443 \cdot lgt$$



Регрессия данных в Python

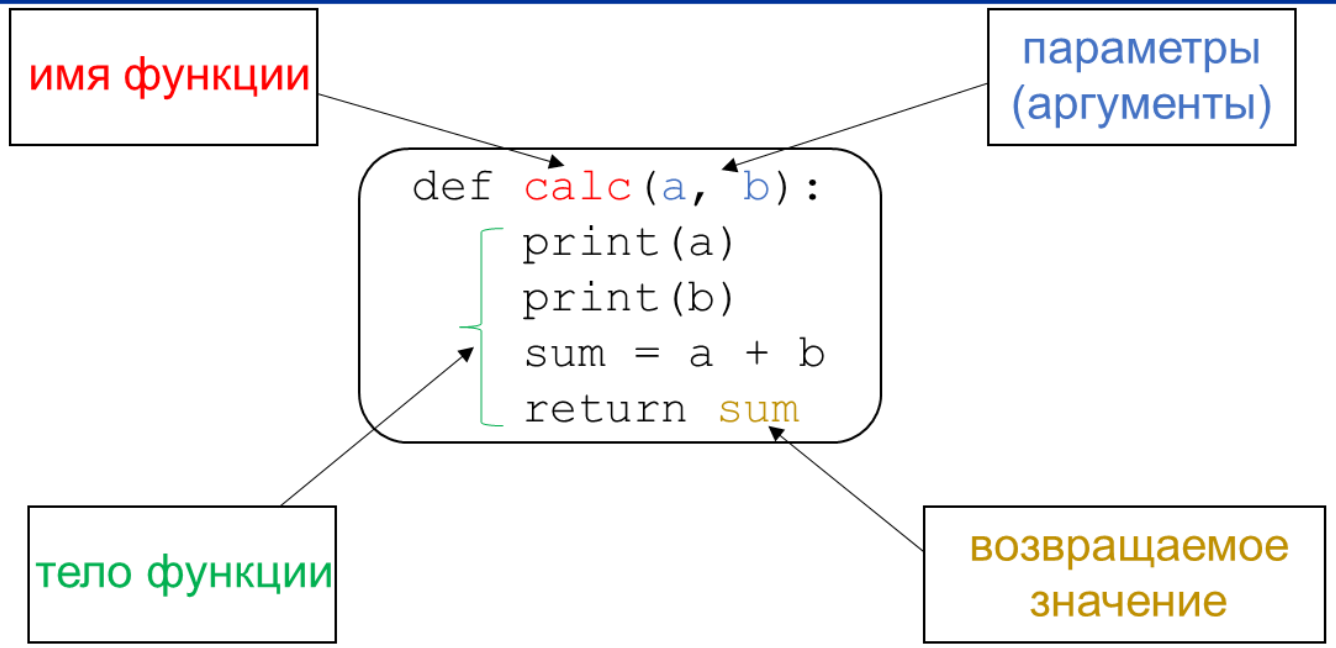
✓ Библиотеки Python для линейной и нелинейной регрессии:

- **NumPy** — это основная библиотека Python для научных вычислений. Позволяет выполнять множество операций с массивами и множество других математических операций.
- **SciPy** — библиотека для научных вычислений, включающая инструменты для оптимизации, интегрирования, интерполяции и статистики.
- **scikit-learn** — это библиотека Python для машинного обучения, созданная поверх NumPy и некоторых других пакетов. Содержит средства для предварительной обработки данных, уменьшения размерности, реализации регрессии, классификации, кластеризации и многого другого.
- **statsmodels**. Если вы хотите реализовать регрессию и нуждаетесь в функциональности, выходящей за рамки scikit-learn, вам следует рассмотреть эту библиотеку. Используется для оценки статистических моделей.

Алгоритм решения задачи регрессии с помощью Python

- Импортируйте нужные библиотеки
- Введите/импортируйте данные для работы и выполните необходимые преобразования данных
- Создайте модель регрессии и сопоставьте ее с реальными данными
- Проверьте адекватность модели

Функции в Python



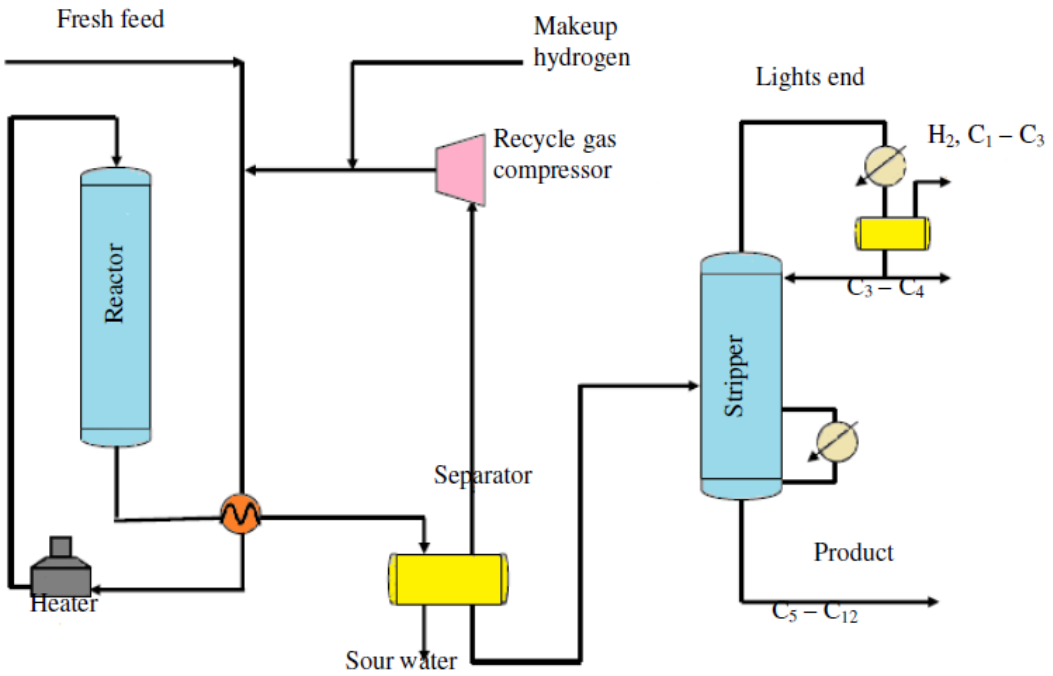
Пример вызова функции:

```
result = calc(3, 5)
print("Сумма a+b:", result)
```

Результат:

```
3
5
Сумма a+b: 8
```

Пример: гидроочистка бензиновой фракции





- Как скорость реакции гидроочистки бензиновой фракции зависит от давления ВСГ? Можно предположить, что скорость реакции описывается уравнением вида

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

Гидроочистка бензиновой фракции: нелинейная регрессия с помощью МНК (библиотека SciPy)

r, моль/кг/с	P, МПа
0.0192	1.382
0.0234	2.075
0.0258	2.431
0.0281	2.762
0.0318	3.5
0.0319	3.833
0.0334	4.129

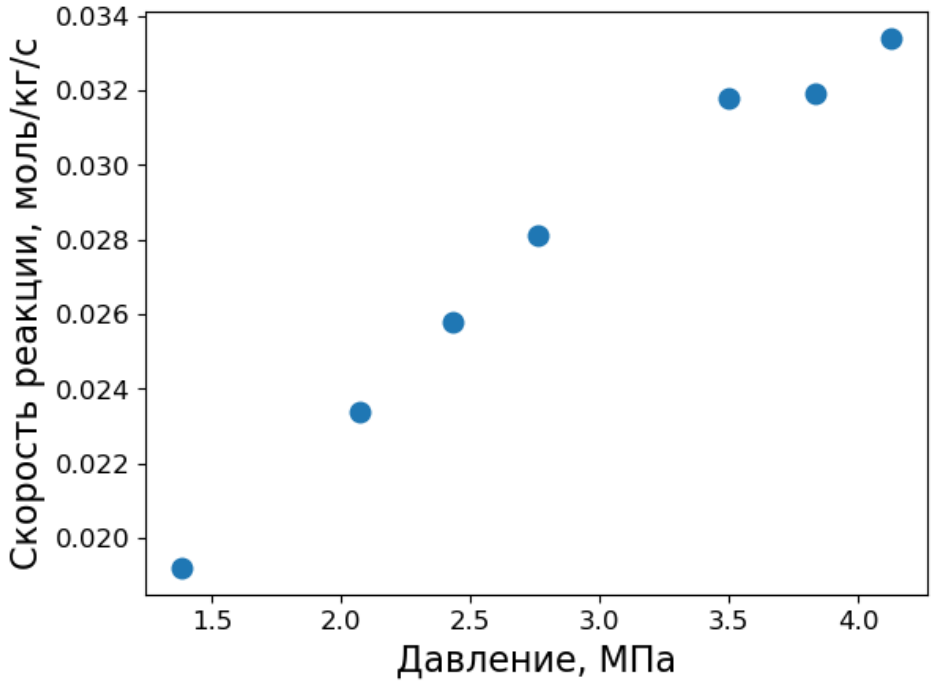
```
# Импортируем библиотеки
from scipy.optimize import curve_fit
import numpy as np
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt

# Вводим исходные данные
r = np.array([0.0192, 0.0234, 0.0258, 0.0281, 0.0318, 0.0319, 0.0334])
P = np.array([1.382, 2.075, 2.431, 2.762, 3.5, 3.833, 4.129])
```

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

Гидроочистка бензиновой фракции: нелинейная регрессия с помощью МНК (библиотека SciPy)

r, моль/кг/с	P, МПа
0.0192	1.382
0.0234	2.075
0.0258	2.431
0.0281	2.762
0.0318	3.5
0.0319	3.833
0.0334	4.129




```python
# Импортируем библиотеки
from scipy.optimize import curve_fit
import numpy as np
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt

# Вводим исходные данные
r = np.array([0.0192, 0.0234, 0.0258, 0.0281, 0.0318, 0.0319, 0.0334])
P = np.array([1.382, 2.075, 2.431, 2.762, 3.5, 3.833, 4.129])

# Строим график экспериментальных данных
plt.plot(P, r, 'o', markersize = 9)
plt.xlabel('Давление, МПа', fontsize = 16)
plt.ylabel('Скорость реакции, моль/кг/с', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.savefig('rp_plot_exp.png', dpi = 300)
plt.show()
```

$$r(P) = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

Основные параметры curve_fit() из scipy.optimize:

Обязательные:

1) **f** — функция, в которой содержится формула, для которой необходимо найти коэффициенты
2) **xdata** — массив значений x (входные данные)
3) **ydata** — массив значений y (выходные данные)

Дополнительные:

p0 — начальное приближение параметров: p0 = [значение_параметра_#1, значение_параметра_#2, …]. Если не указано, то начальные значения будут равны 1.

bounds — границы параметров: (нижняя_граница, верхняя_граница). По умолчанию параметры могут принимать любые значения (-∞, ∞). *Пример в случае двух коэффициентов: bounds = ([0, -np.inf], [10, np.inf]): 1-ый коэффициент ограничен между 0 и 10, второй – может быть любым* (-∞, ∞).

Гидроочистка бензиновой фракции: нелинейная регрессия с помощью МНК (библиотека SciPy)

```
# Определяем математическую форму модели
def f(P, theta1, theta2):
    return theta1 * P / (1 + theta2 * P)

# Находим коэффициенты модели
coeff, coeff_cov = curve_fit(f, P, r)
print('Коэффициенты модели: ', coeff)
```

$$r = \frac{\theta_1 \cdot P}{1 + \theta_2 \cdot P}$$

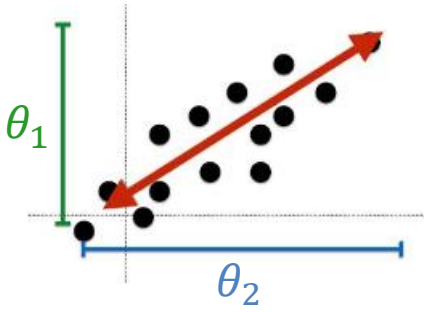
Коэффициенты модели: [0.02051529 0.37325111]

```
# Определяем стандартное отклонение
# (ошибку в определении)
# каждого коэффициента (тета1 и тета2)
std_dev = np.sqrt(np.diag(coeff_cov))
print('Ошибка определения коэффициента: ', std_dev)
```

Ошибка определения коэффициента: [0.00104223 0.03533892]

```
# Вычисляем модельные значения скорости реакции
r_model = f(P, coeff[0], coeff[1])
print(r_model)
```

[0.01870399 0.02398947 0.0261473 0.02790028 0.03113257 0.03235118 0.03333431]



coeff_cov:

	θ_1	θ_2
θ_1	$var(\theta_1)$	$cov(\theta_1, \theta_2)$
θ_2	$cov(\theta_1, \theta_2)$	$var(\theta_2)$

$$cov(\theta_1, \theta_2) = \sum_i \frac{(\theta_1^i - \overline{\theta_1}) \cdot (\theta_2^i - \overline{\theta_2})}{n}$$

$$var(\theta_1) = \sum_i \frac{(\theta_1^i - \overline{\theta_1})^2}{n}$$

$$var(\theta_2) = \sum_i \frac{(\theta_2^i - \overline{\theta_2})^2}{n}$$

$$std_dev(\theta_1) = \sqrt{var(\theta_1)}$$

$$std_dev(\theta_2) = \sqrt{var(\theta_2)}$$

Гидроочистка бензиновой фракции: нелинейная регрессия с помощью МНК (библиотека SciPy)

```
# Определяем адекватность модели,
# вычислив коэффициент детерминации
r_squared = r2_score(r, r_model)  или  r_squared = r2_score(r, f(P, coeff[0], coeff[1]))
print('R2: ', r_squared)
```

R2: 0.9912823889977744

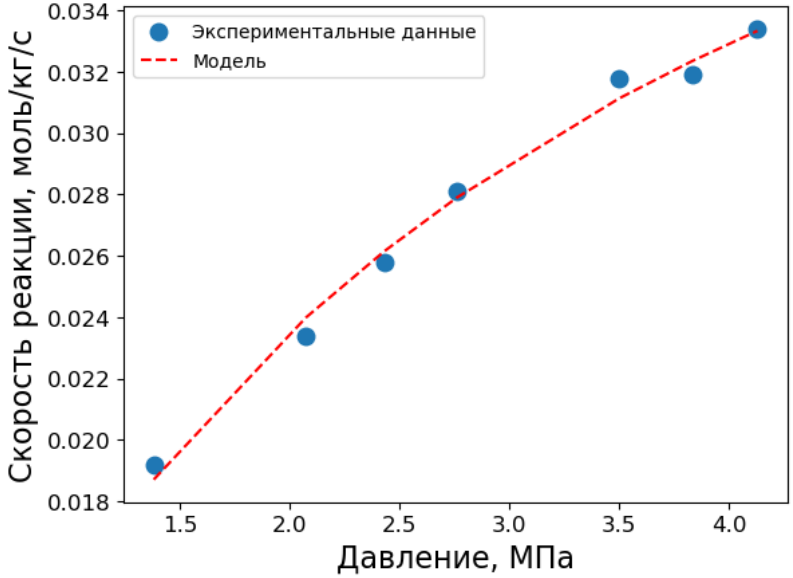
```
# Строим график
plt.plot(P, r, 'o', markersize = 9, label = 'Экспериментальные данные')
plt.plot(P, r_model, 'r--', label = 'Модель')
plt.legend()
plt.xlabel('Давление, МПа', fontsize = 16)
plt.ylabel('Скорость реакции, моль/кг/с', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.savefig('rp_plot_curve_fit.png', dpi = 300)
plt.show()
```

$$r = \frac{0.02052 \cdot P}{1 + 0.37325 \cdot P}$$

$$\theta_1 = 0.02052 \pm 0.00104$$

$$\theta_2 = 0.37325 \pm 0.03534$$

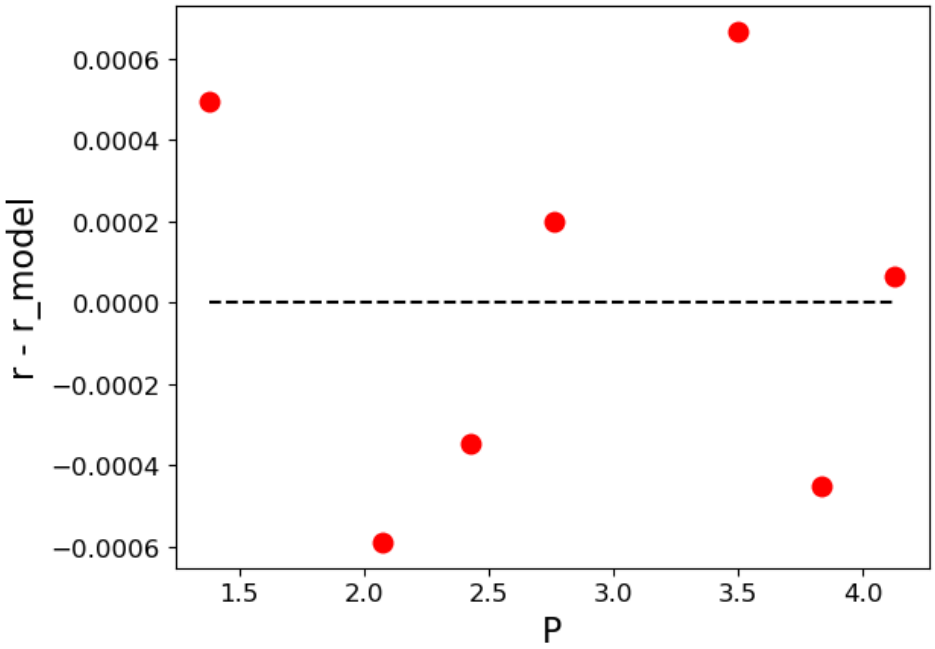
$$R^2 = 0.991$$



Гидроочистка бензиновой фракции: нелинейная регрессия с помощью МНК (библиотека SciPy)

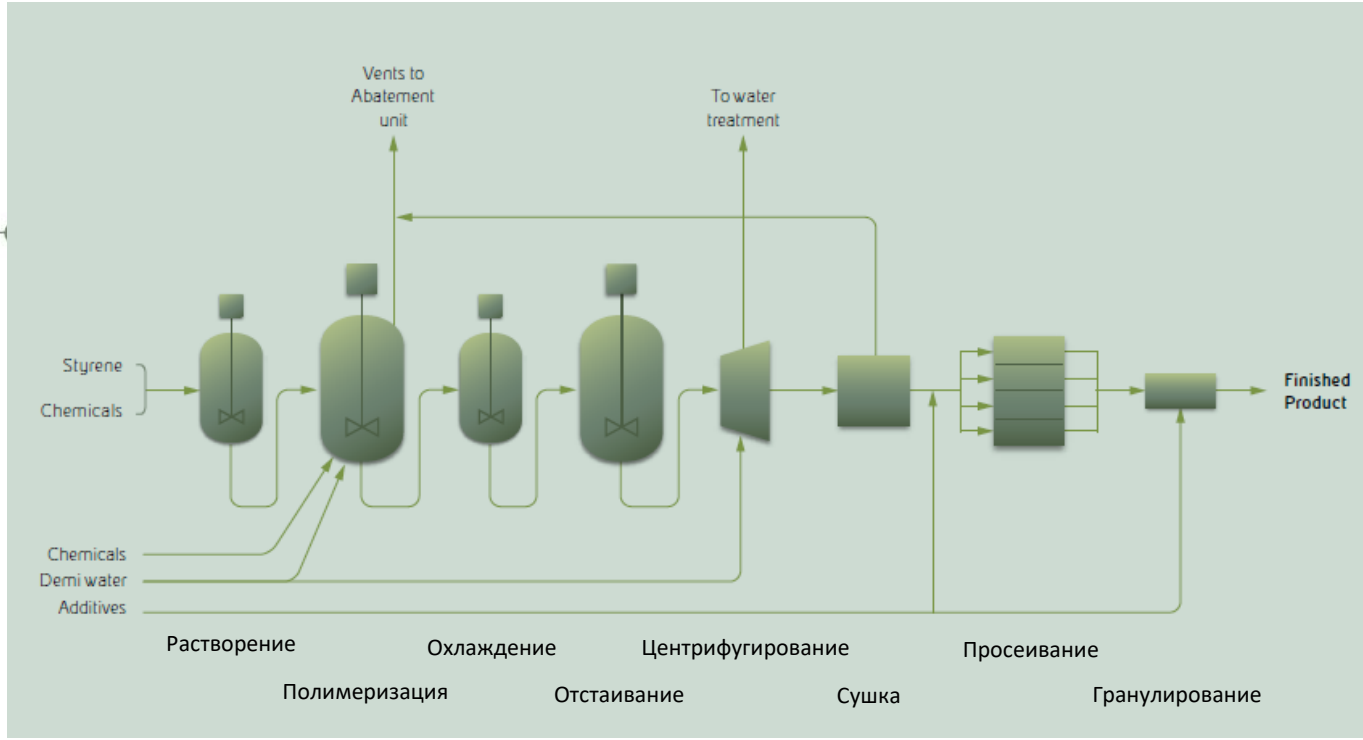
```
# Строим график остатков
plt.plot(P, (r - r_model), 'ro', markersize = 9)
plt.plot(P, (r * 0), 'k--')
plt.xlabel('P', fontsize = 16)
plt.ylabel('r - r_model', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.savefig('rp_residual_plot.png', dpi = 300)
plt.show()
```

График остатков



Пример: производство полистирола







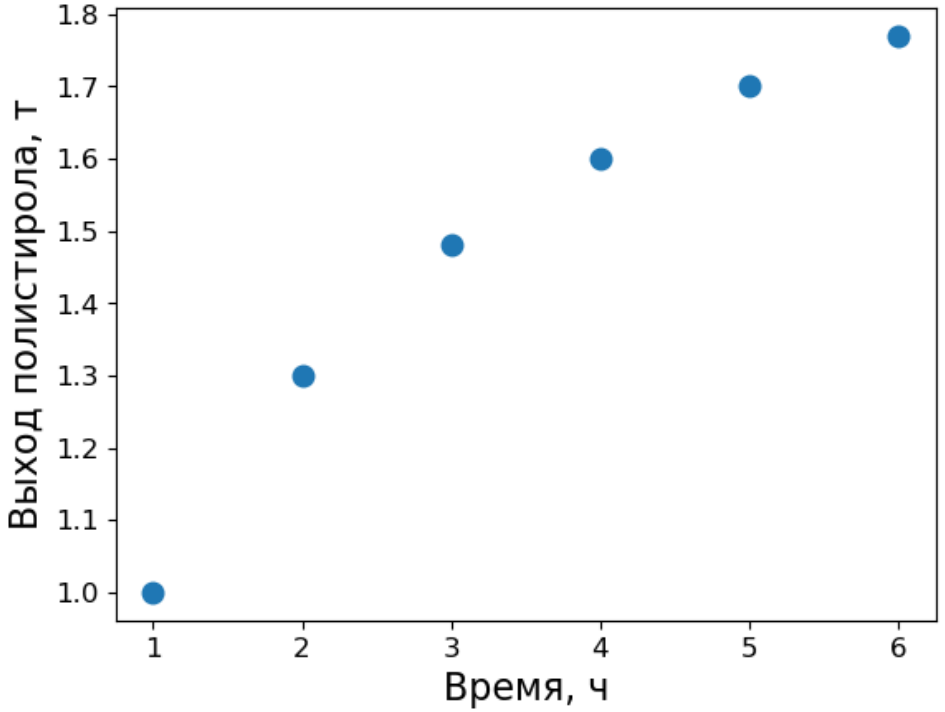


Пример: производство полистирола



- Данные работы установки суспензионной полимеризации стирола – зависимость количества полистирола (q в тоннах) от времени (t в часах):

t, ч	q, т
1	1
2	1.3
3	1.48
4	1.6
5	1.7
6	1.77



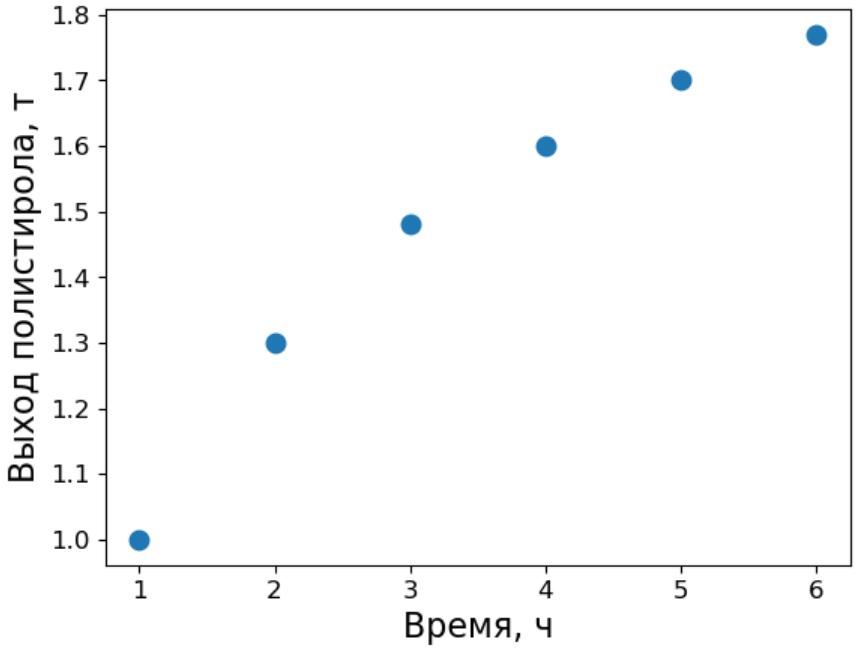
Производство ПС: линейная регрессия с NumPy

t, ч	q, т
1	1
2	1.3
3	1.48
4	1.6
5	1.7
6	1.77

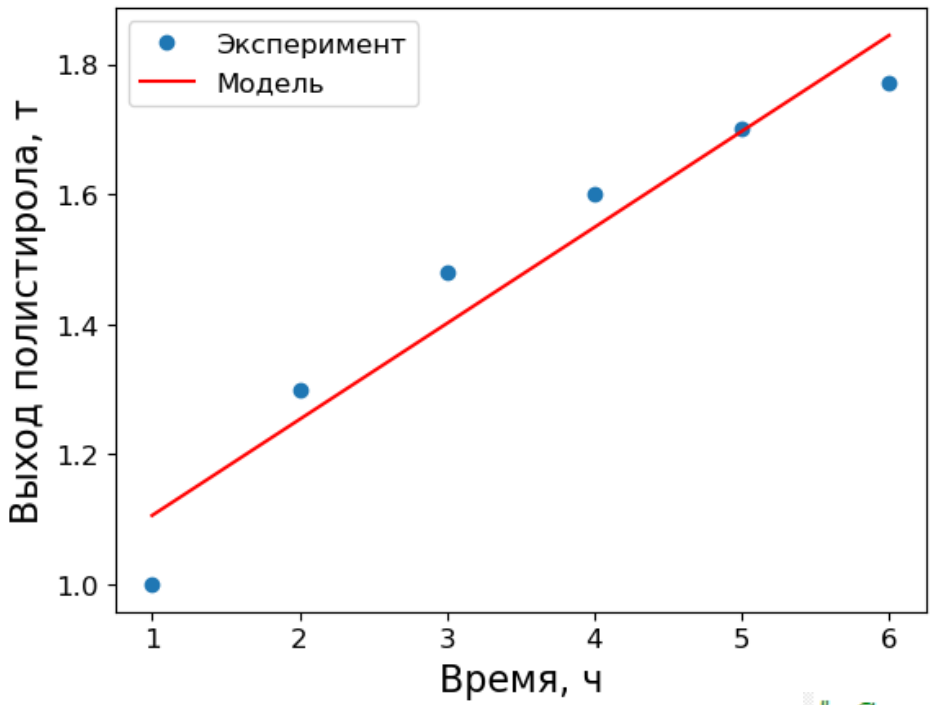
```
# Импортируем библиотеки
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import r2_score

# Вводим исходные данные
t = np.array([1, 2, 3, 4, 5, 6])
q = np.array([1, 1.3, 1.48, 1.6, 1.7, 1.77])

# Строим график экспериментальных данных
plt.plot(t, q,'o', markersize = 9)
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.savefig('qt_plot_exp.png', dpi = 300)
plt.show()
```



Производство ПС: линейная регрессия с NumPy




```
# Находим коэффициенты b0 и b1
# полинома 1ой степени q = b1 * t + b0
p1 = np.polyfit(t, q, 1)
print('b1, b0: ' + str(p1))
```

b1, b0: [0.14771429 0.958]

```
# Вычисляем модельные значения выхода полистирола
q_model = np.polyval(p1, t)
```

```
# Определяем адекватность модели,
# вычислив коэффициент детерминации
r2_coeff = r2_score(q, q_model)
print('R2: ', r2_coeff)
```

R2: 0.932343861729773

$$q = 0.1477 \cdot t + 0.958$$

$$R^2 = 0.9323$$

```
# Строим график
plt.plot(t, q, 'o', label = 'Эксперимент')
plt.plot(t, q_model, 'r-', label = 'Модель')
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.legend(fontsize = 12)
plt.savefig('qt_plot.png', dpi = 300)
```

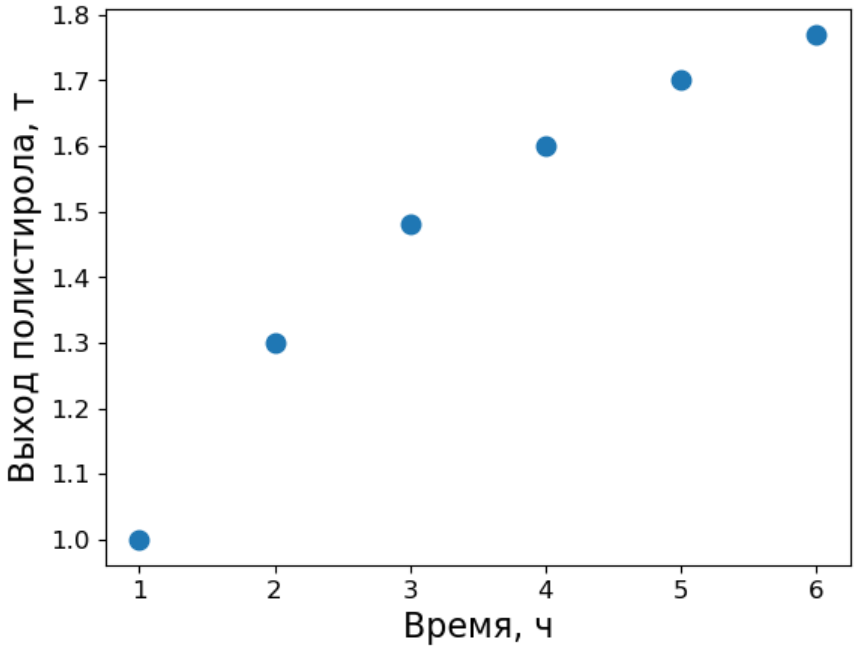
Производство ПС: полиномиальная регрессия с NumPy

t, ч	q, т
1	1
2	1.3
3	1.48
4	1.6
5	1.7
6	1.77

```
# Импортируем библиотеки
import matplotlib.pyplot as plt
import numpy as np

# Вводим исходные данные
t = np.array([1, 2, 3, 4, 5, 6])
q = np.array([1, 1.3, 1.48, 1.6, 1.7, 1.77])

# Строим график экспериментальных данных
plt.plot(t, q,'o', markersize = 9)
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.savefig('qt_plot_exp.png', dpi = 300)
plt.show()
```



Производство ПС: полиномиальная регрессия с NumPy

```
# Находим коэффициенты
# полиномов n-ой степени
p1 = np.polyfit(t, q, 1)
p2 = np.polyfit(t, q, 2)
p3 = np.polyfit(t, q, 3)
p4 = np.polyfit(t, q, 4)
print('Модель n=1: ', p1)
print('Модель n=2: ', p2)
print('Модель n=3: ', p3)
print('Модель n=4: ', p4)
```

```
Модель n=1:  [0.14771429  0.958]
Модель n=2:  [-0.02625  0.33146429  0.713]
Модель n=3:  [ 0.00527778  -0.08166667
0.49876984  0.58]
Модель n=4:  [-0.00145833  0.02569444
-0.17895833  0.67960317  0.475]
```

$$q = 0.148 \cdot t + 0.958$$

$$q = -0.027 \cdot t^2 + 0.331 \cdot t + 0.713$$

$$q = 0.0053 \cdot t^3 - 0.082 \cdot t^2 + 0.499 \cdot t + 0.58$$

$$q = -0.0015 \cdot t^4 + 0.0257 \cdot t^3 - 0.179 \cdot t^2 + 0.680 \cdot t + 0.475$$

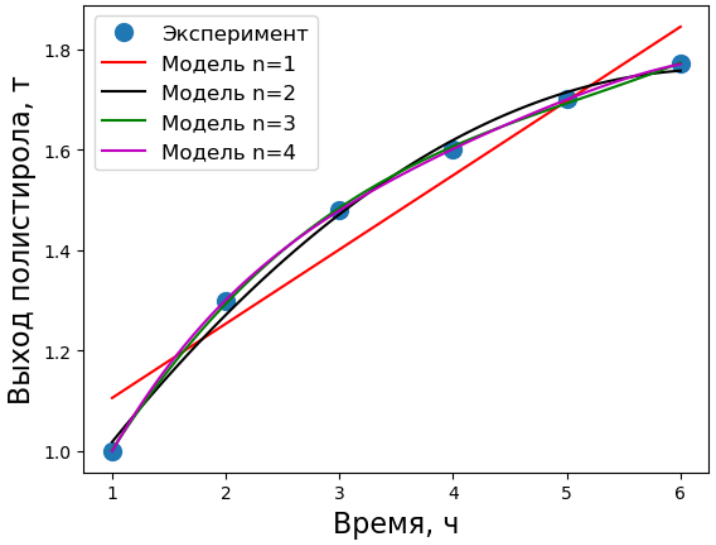
Производство ПС: полиномиальная регрессия с NumPy

$$R^2 = 1 - \frac{SSE}{TSS} = 1 - \frac{\sum_{i=1}^{n}\left(y_i^{exp} - y_i^{model}\right)^2}{\sum_{i=1}^{n}\left(y_i^{exp} - \overline{y^{exp}}\right)^2} = 1 - \frac{\sum_{i=1}^{n}\left(y_i^{exp} - y_i^{model}\right)^2}{n \cdot \sigma^2}$$

```
# Определяем адекватность модели,
# вычислив коэффициент детерминации
# Пример для полинома 1ой степени
q_model1 = p1[0] * t + p1[1]
RSS = np.sum((q - q_model1) ** 2)
TSS = len(q) * np.var(q)    ИЛИ    TSS = np.sum((q - np.mean(q)) ** 2)
Rsq = 1 - RSS / TSS
print('R2 для модели n=1: ', Rsq)
```

R2 для модели n=1: 0.932343861729773

```
# Строим график
t_model = np.linspace(1, 6, 100)
plt.plot(t, q, 'o', markersize = 10, label = 'Эксперимент')
plt.plot(t_model, np.polyval(p1, t_model), 'r-', label = 'Модель n=1')
plt.plot(t_model, np.polyval(p2, t_model), 'k-', label = 'Модель n=2')
plt.plot(t_model, np.polyval(p3, t_model), 'g-', label = 'Модель n=3')
plt.plot(t_model, np.polyval(p4, t_model), 'm-', label = 'Модель n=4')
plt.legend(fontsize = 12)
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.savefig('qt_plot.png', dpi = 300)
plt.show()
```



Производство ПС: линейная регрессия с scikit-learn

```
# Импортируем библиотеки
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# LinearRegression – класс, который позволяет выполнять линейную и полиномиальную регрессию
```

✓ Вводим и подготавливаем данные для построения модели

```
# Вводим исходные данные
# и выполняем необходимые преобразования

t = np.array([1, 2, 3, 4, 5, 6])
print('Форма массива: ', t.shape)
print('Размерность массива: ', t.ndim)
print('Исходный массив t: ', t)
```

Форма массива: (6,)
Размерность массива: 1
Исходный массив t: [1 2 3 4 5 6]

```
t = t.reshape(-1, 1)
# Преобразование формы массива
# Если длина измерения установлена на -1,
# то эта длина определяется автоматически
# Это полезно при работе с большим массивами
print('Преобразованная форма массива: ', t.shape)
print('Преобразованная размерность массива: ', t.ndim)
print('Преобразованный массив t: ', t)

q = np.array([1, 1.3, 1.48, 1.6, 1.7, 1.77])
```

Преобразованная форма массива: (6, 1)
Преобразованная размерность массива: 2
Преобразованный массив t: [[1]
[2]
[3]
[4]
[5]
[6]]

Производство ПС: линейная регрессия с scikit-learn

```
model = LinearRegression() # Создаем пустой объект класса
```

Параметры:
fit_intercept — *если True (по умолчанию), то определяет точку пересечения с осью OY. Если False, то считает ее равной 0*
copy_X — *решает, следует копировать (True, по умолчанию) или перезаписывать (False) входные данные (False)*
n_jobs — *на скольких процессорах параллельно решать задачу*
positive — *если False (по умолчанию), то коэффициенты могут быть как положительные, так и отрицательные. Если True, то все коэффициенты будут ≥ 0)*

```
# Находим коэффициенты b1 и b0 линейной модели q = b1*t + b0
model.fit(t, q)
print('b0: ', model.intercept_)
print('b1: ', model.coef_)
```

b0: 0.9579999999999997
b1: [0.14771429]

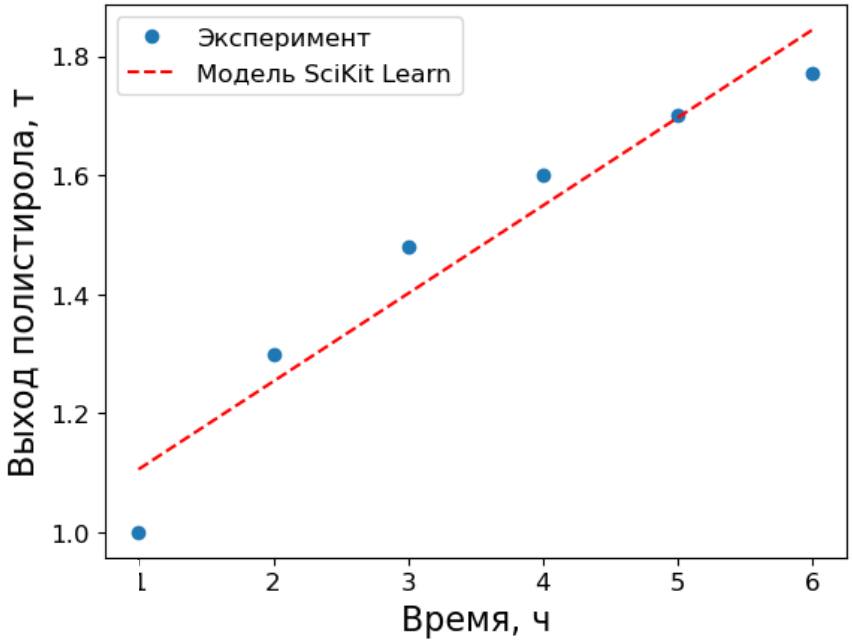
Производство ПС: линейная регрессия с scikit-learn

```
# Определяем адекватность модели, вычислив коэффициент детерминации
print('R2: ', model.score(t, q))
```

R2: 0.932343861729773

```
# Вычисляем модельные значения выхода полистирола
q_model = model.predict(t)
print(q_model)
```

[1.10571429 1.25342857 1.40114286
1.54885714 1.69657143 1.84428571]




```
# Строим график
plt.plot(t, q, 'o', label = 'Эксперимент')
plt.plot(t, q_model, 'r--', label = 'Модель SciKit Learn')
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.legend(fontsize = 12)
plt.savefig('qt_plot_scikit.png', dpi = 300)
plt.show()
```

$$q = 0.1477 \cdot t + 0.958$$

$$R^2 = 0.9323$$

Производство ПС: полиномиальная регрессия с scikit-learn

```
# Импортируем библиотеки
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

✓ Вводим и подготавливаем данные для построения модели

```
# Входные данные должны быть двумерным массивом
t = np.array([1, 2, 3, 4, 5, 6]).reshape((-1, 1))
q = np.array([1, 1.3, 1.48, 1.6, 1.7, 1.77])
print('t: ', t)

# Преобразуем входные данные для полиномиальной регрессии
# y = b0 + b1 * x + b2 * x ^ 2 + ... + bn * x ^ n

t_ = PolynomialFeatures(degree = 2, include_bias = False).fit_transform(t)

# Параметры:
# degree - степень полинома - это целое число (по умолчанию 2).
# interaction_only - определяет все (False, по умолчанию) или только смешанные (True) члены полинома
# используются для построения модели
# y = b0 + b10 * x1 + b01 * x2 + b11 * x1 ^ 2 + b22 * x2 ^ 2 + b12 * x1 * x2 +... (все)
# y = b0 + b10 * x1 + b01 * x2 + b12 * x1 * x2 +... (только смешанные)
# include_bias - Если True (по умолчанию), то добавляется дополнительный столбец,
# в котором все степени в полиноме равны нулю (т. е. столбец из единиц - аналог b0 в линейной модели)

print('t_: ', t_)
```

```
t: [[1]
 [2]
 [3]
 [4]
 [5]
 [6]]
t_: [[ 1.  1.]
 [ 2.  4.]
 [ 3.  9.]
 [ 4. 16.]
 [ 5. 25.]
 [ 6. 36.]]
```

Производство ПС: полиномиальная регрессия с scikit-learn

✓ Создаем модель и определяем ее параметры

```
# Создаем модель 2ой степени
model2 = LinearRegression().fit(t_, q)

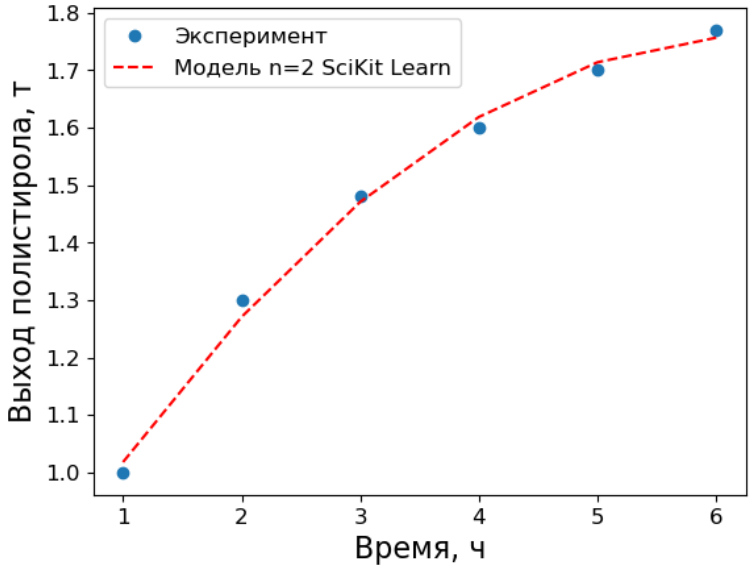
# Определяем параметры модели
# q = b2 * t ^ 2 + b1 * t + b0
print('b0: ', model2.intercept_)
print('b1, b2: ', model2.coef_)

# Определяем адекватность модели,
# вычислив коэффициент детерминации
print('R2: ', model2.score(t_, q))
```

```
# Вычисляем модельные значения выхода полистирола
q_model2 = model2.predict(t_)
print(q_model2)
```

[1.01821429 1.27092857 1.47114286
1.61885714 1.71407143 1.75678571]

```
# Строим график
plt.plot(t, q, 'o', label = 'Эксперимент')
plt.plot(t, q_model2, 'r--', label = 'Модель n=2 SciKit Learn')
plt.xlabel('Время, ч', fontsize = 16)
plt.ylabel('Выход полистирола, т', fontsize = 16)
plt.xticks(fontsize = 12)
plt.yticks(fontsize = 12)
plt.legend(fontsize = 12)
plt.savefig('qt_plot_scikit_polynom.png', dpi = 300)
plt.show()
```



$$q = -0.02625 \cdot t^2 + 0.3315 \cdot t + 0.713$$

b0: 0.7129999999999997
b1, b2: [0.33146429 -0.02625]
R2: 0.9951567050944417